

云原生训练营

模块八- Kubernetes 控制平面组件： 生命周期管理和服务发现

孟凡杰

前 eBay 资深架构师

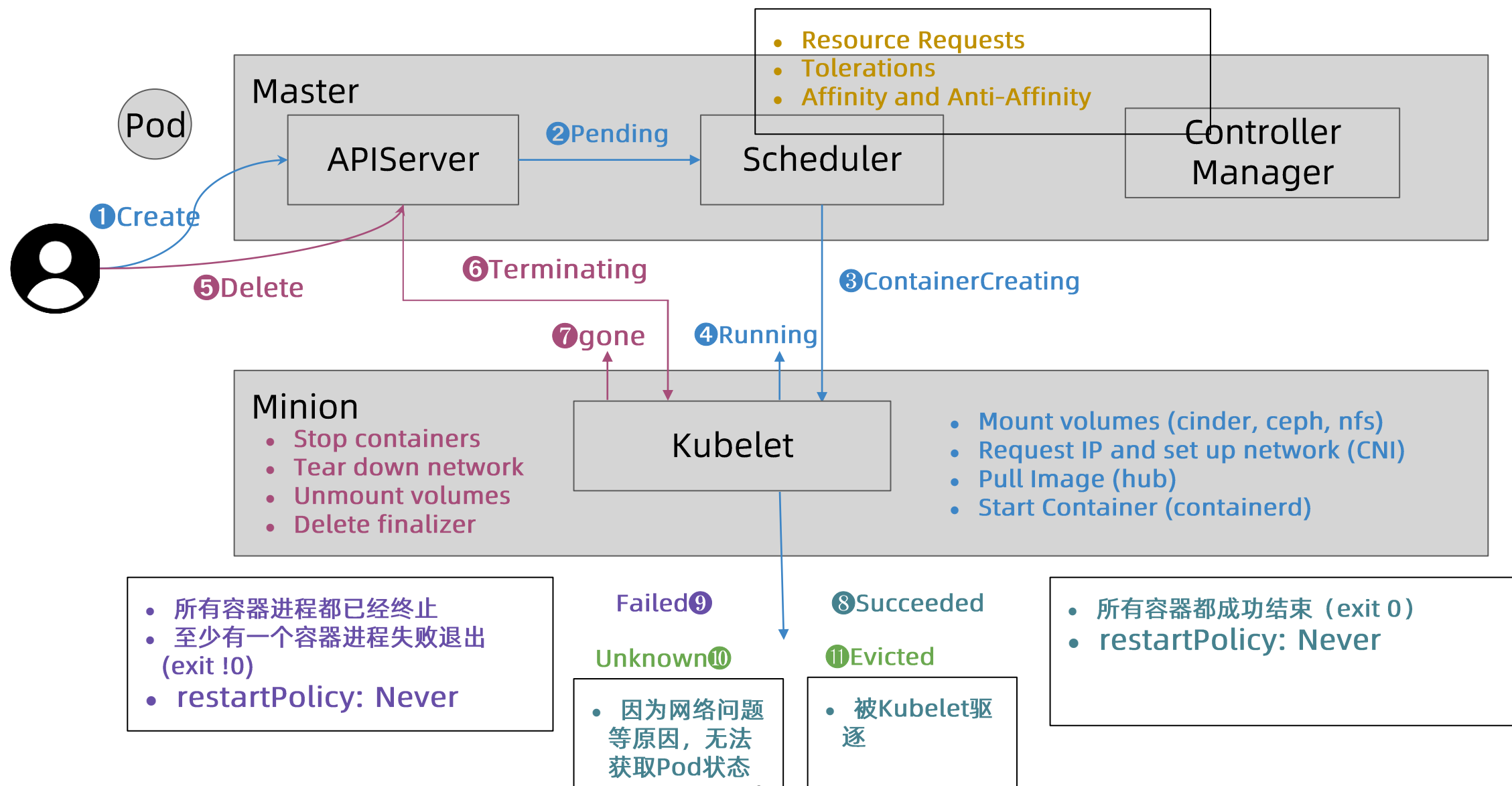


目录

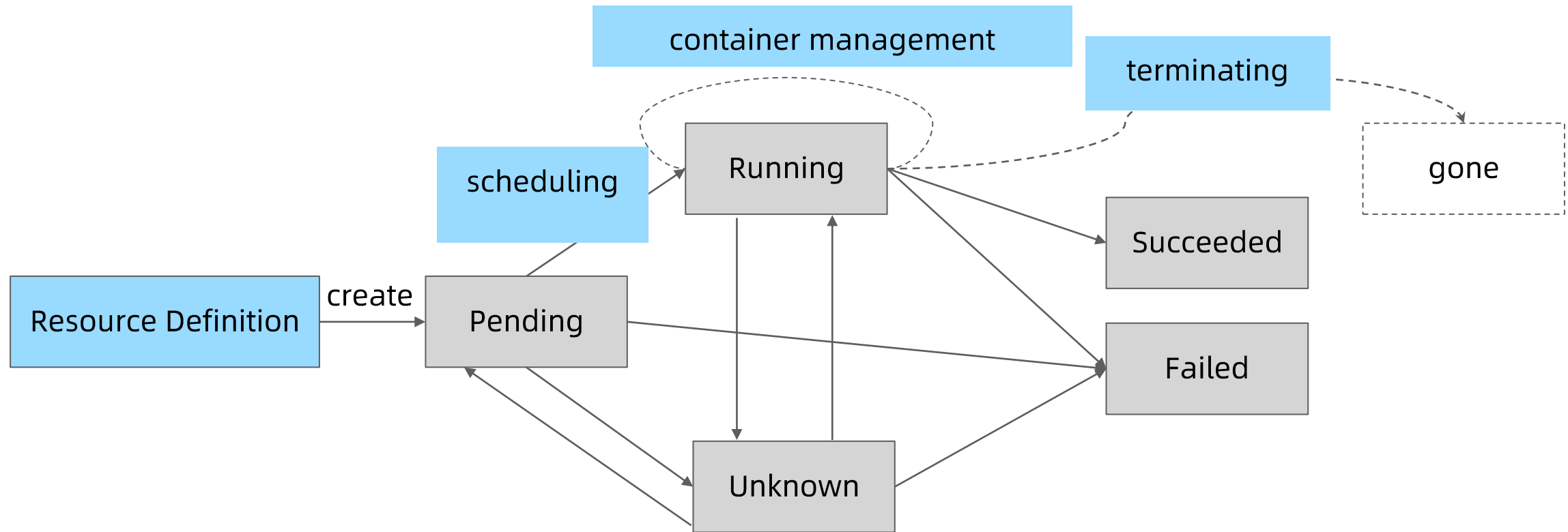
- 深入理解Pod的生命周期
- 服务发现
- 微服务架构下的高可用挑战
- Service对象
- kube-proxy

1. 深入理解Pod的生命周期

如何优雅的管理Pod的完整生命周期



Pod状态机



Pod Phase

Pod Phase

- Pending
- Running
- Succeeded
- Failed
- Unknown

kubectl get pod显示的状态信息是由podstatus的conditions和phase计算出来的

- 查看pod细节

kubectl get pod \$podname -oyaml

- 查看pod相关事件

kubectl describe pod

```
status:
  conditions:
  - status: "True"
    type: Initialized
  - message: 'containers with unready status: [netd]'
    reason: ContainersNotReady
    status: "False"
    type: Ready
  - message: 'containers with unready status: [netd]'
    reason: ContainersNotReady
    status: "False"
    type: ContainersReady
  - status: "True"
    type: PodScheduled
  containerStatuses:
  - containerID: containerd://05357cc
    lastState:
      terminated:
        containerID: containerd://*
        exitCode: 128
        finishedAt: "2020-07-12T05:53:36Z"
        message: 'failed to create containerd task: failed
          failed to create and open log file: open /var/lo
          disk quota exceeded'
        reason: StartError
        startedAt: "1970-01-01T00:00:00Z"
    name: netd
    ready: false
    restartCount: 4298
    state:
      waiting:
        message: Back-off 5m0s restarting failed container
        reason: CrashLoopBackOff
  initContainerStatuses:
  - containerID: containerd://*
    name: netd-init
    ready: true
    restartCount: 1
    state:
      terminated:
        containerID: containerd://36fd8ff2326ae4f639c2f3f4
        exitCode: 0
        finishedAt: "2020-06-27T03:08:25Z"
        reason: Completed
        startedAt: "2020-06-27T03:08:22Z"
  phase: Running
  podIP: 10.148.140.46
  qosClass: Burstable
```

Pod状态计算细节

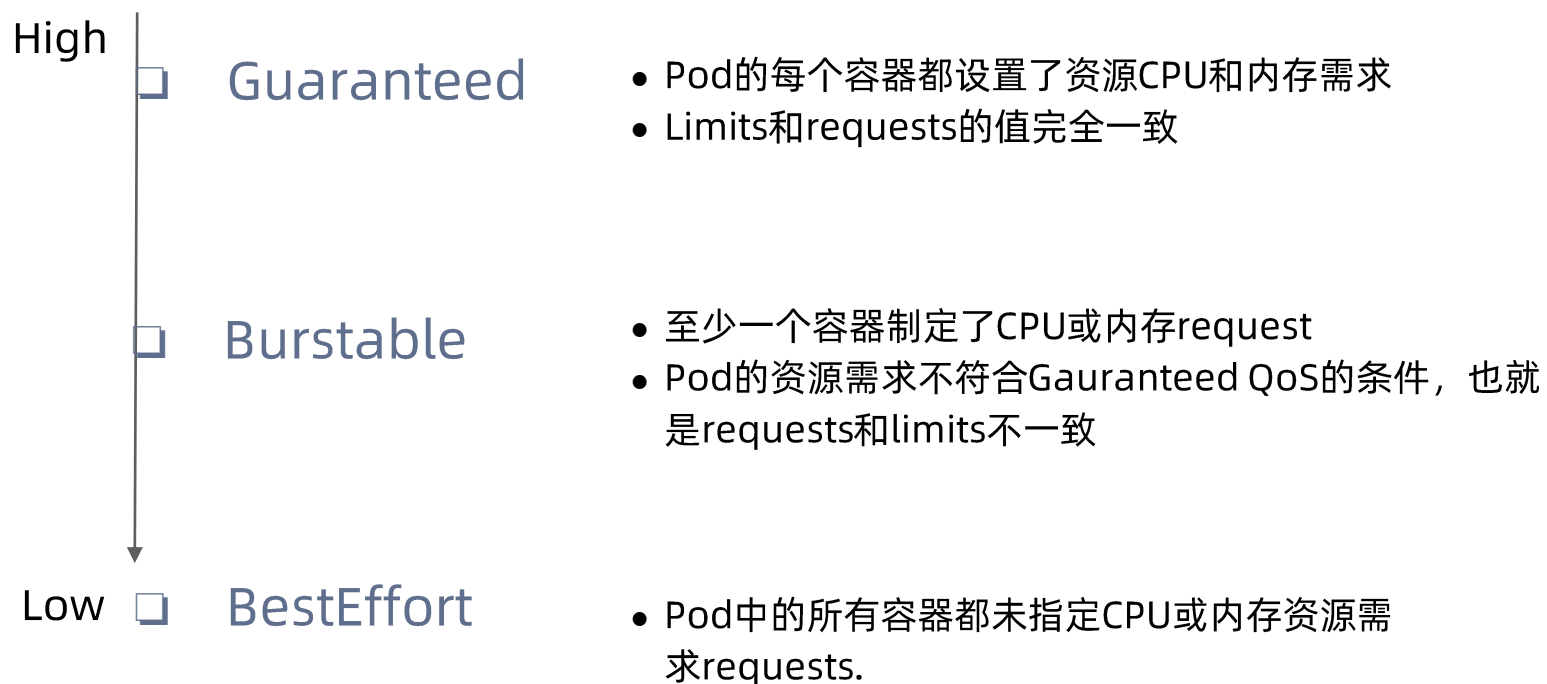
kubectl get pod返回的状态	Pod Phase	Conditions
Completed	Succeeded	
ContainerCreating	Pending	
CrashLoopBackOff	Running	Container exits
CreateContainerConfigError	Pending	configmap " test" not found secret " my-secret" not found
ErrImagePull / ImagePullBackOff / Init:ImagePullBackOff / InvalidImageName	Pending	Back-off pulling image
Error	Failed	restartPolicy: Never container exits with Error (not 0)
Evicted	Failed	message: 'Usage of EmptyDir volume "myworkdir" exceeds the limit "40Gi". ' reason: Evicted
Init:0/1	Pending	Init containers don't exit
Init:CrashLoopBackOff / Init>Error	Pending	Init container crashed (exit with not 1)
OOMKilled	Running	Containers are OOMKilled
StartError	Running	Containers cannot be started
Unknown	Running	Node NotReady
OutOfCpu/OutOfMemory	Failed	Scheduled, but it cannot pass kubelet admit

如何确保Pod的高可用

- 避免容器进程被终止避免Pod被驱逐
 - 设置合理的 `resources.memory limits` 防止容器进程被OOMKill
 - 设置合理的 `emptydir.sizeLimit`并且确保数据写入不超过emptyDir的限制，防止Pod被驱逐

Pod的QoS分类

- [QoS & Quota] Guaranteed, Burstable and BestEffort



```
spec:
  containers:
    ...
    resources:
      limits:
        cpu: 700m
        memory: 200Mi
      requests:
        cpu: 700m
        memory: 200Mi
    ...
  qosClass: Guaranteed
```

```
spec:
  containers:
    ...
    resources:
      limits:
        memory: 200Mi
      requests:
        memory: 100Mi
    ...
  qosClass: Burstable
```

```
spec:
  containers:
    ...
    resources: {}
    ...
  qosClass: BestEffort
```

当计算节点检测到内存压力时，Kubernetes会按BestEffort-> Burstable-> Guaranteed的顺序依次驱逐Pod

```
$ kubectl get pod test-vfagg6 -oyaml |grep qosClass
qosClass: Burstable
```

质量保证Guaranteed, Burstable and BestEffort

定义Guaranteed类型的资源需求来保护你的重要Pod.

认真考量Pod需要的真实需求并设置limit和resource，这有利于将集群资源利用率控制在合理范围并减少Pod被驱逐的现象。

尽量避免将生产Pod设置为BestEffort，但是对测试环境来讲，BestEffort Pod能确保大多数应用不会因为资源不足而处于Pending状态。

Burstable 适用于大多数场景。思考：为什么？

基于Taint的Evictions

NotReady Node

taints:

- effect: NoSchedule
key: node.kubernetes.io/unreachable
timeAdded: "2020-07-09T11:25:10Z"
- effect: NoExecute
key: node.kubernetes.io/unreachable
timeAdded: "2020-07-09T11:25:21Z"
- effect: NoSchedule
key: node.kubernetes.io/not-ready
timeAdded: "2020-07-09T11:24:28Z"
- effect: NoExecute
key: node.kubernetes.io/not-ready
timeAdded: "2020-07-09T11:24:32Z"

K8s为Pod自动增加的Toleration

tolerations:

- effect: NoExecute
key: node.kubernetes.io/not-ready
operator: Exists
tolerationSeconds: 900
- effect: NoExecute
key: node.kubernetes.io/unreachable
operator: Exists
tolerationSeconds: 900

- 节点临时不可达
 - 网络分区
 - kubelet, containerd不工作
 - 节点重启超过了15分钟
- 增大tolerationSeconds 以避免被驱逐
 - 特别是依赖于本地存储状态的有状态应用

健康检查探针

- 健康探针类型分为
 - livenessProbe
 - 探活，当检查失败时，意味着该应用进程已经无法正常提供服务，kubelet会终止该容器进程并按照restartPolicy决定是否重启
 - readinessProbe
 - 就绪状态检查，当检查失败时，意味着应用进程正在运行，但因为某些原因不能提供服务，Pod状态会被标记为NotReady
 - startupProbe
 - 在初始化阶段（Ready之前）进行的健康检查，通常用来避免过于频繁的监测影响应用启动
- 探测方法包括
 - ExecAction: 在容器内部运行指定命令，当返回码为0时，探测结果为成功
 - TCPSocketAction: 由kubelet发起，通过TCP协议检查容器IP和端口，当端口可达时，探测结果为成功
 - HTTPGetAction: 由kubelet发起，对Pod的IP和指定端口以及路径进行HTTPGet操作，当返回码为200-400之间时，探测结果为成功

```
livenessProbe:
  httpGet:
    path: /healthz
    port: 8080
  httpHeaders:
    - name: X-Custom-Header
      value: Awesome
  initialDelaySeconds: 15
  timeoutSeconds: 1
```

探针属性

parameters	Description
initialDelaySeconds	Defaults to 0 seconds. Minimum value is 0.
periodSeconds	Default to 10 seconds. Minimum value is 1.
timeoutSeconds	Defaults to 1 second. Minimum value is 1.
successThreshold	Defaults to 1. Must be 1 for liveness. Minimum value is 1.
failureThreshold	Defaults to 3. Minimum value is 1.

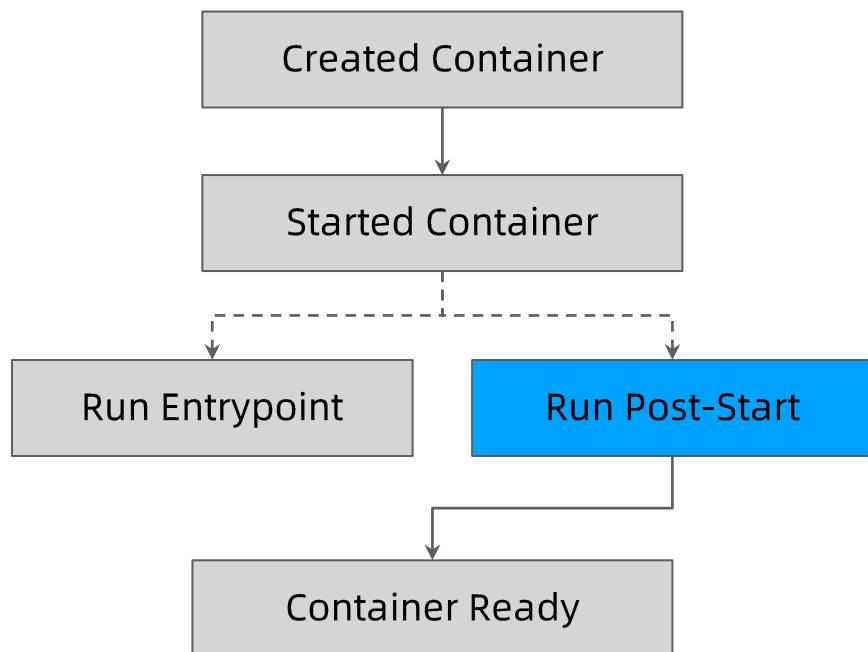
ReadinessGates

- Readiness允许在Kubernetes自带的Pod Conditions之外引入自定义的就绪条件
- 新引入的readinessGates condition需要为True状态后，加上内置的Conditions，Pod才可以以为就绪状态
- 该状态应该由某控制器修改

```
kind: Pod
...
spec:
  readinessGates:
    - conditionType: "www.example.com/feature-1"
status:
  conditions:
    - type: Ready
      status: "False"
      lastProbeTime: null
      lastTransitionTime: 2018-01-01T00:00:00Z
    - type: "www.example.com/feature-1"
      status: "False"
      lastProbeTime: null
      lastTransitionTime: 2018-01-01T00:00:00Z
    - lastProbeTime: null
      lastTransitionTime: "2020-06-23T10:02:57Z"
      status: "True"
      type: ContainersReady
  containerStatuses:
    - containerID: ...
      ready: true
```

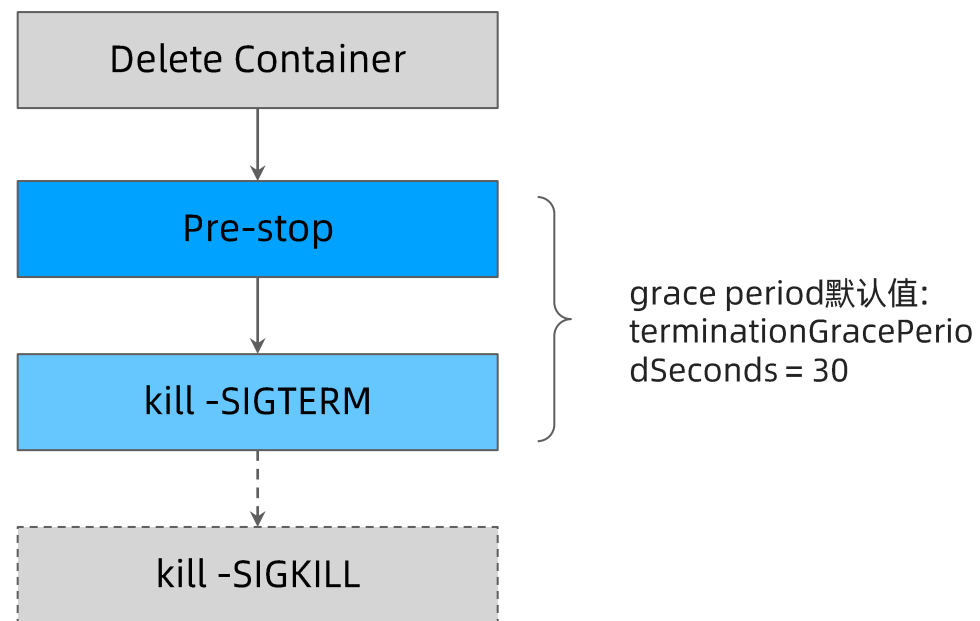
Post-start和Pre-Stop Hook

kind: Pod
spec:
terminationGracePeriodSeconds: 30



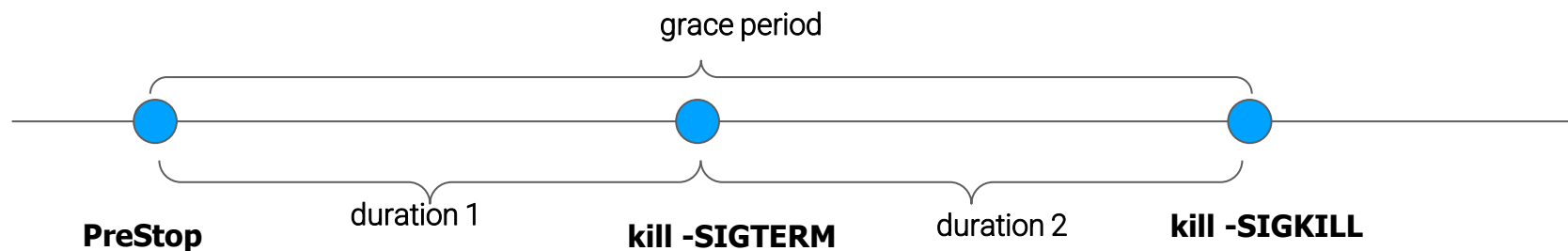
postStart结束之前，容器不会被标记为running状态。

无法保证postStart脚本和容器的Entrypoint哪个先执行



只有当Pod被终止时，Kubernetes才会执行preStop脚本，这意味着当Pod完成或容器退出时，preStop脚本不会被执行。

terminationGracePeriodSeconds的分解



Terminating Pod的误用

spec:
containers:
- image: netperf:0.1
command:
- /netperf-server.sh
imagePullPolicy: Always

\$ cat netperf-server.sh
#!/bin/bash
netserver -D

bash/sh 会忽略SIGTERM信号量，因此kill -SIGTERM 会永远超时，若应用使用bash/sh作为Entrypoint，则应避免过长的grace period

Time Taken by PreStop (duration 1)	Time Taken by kill -SIGTERM (duration 2)	Total Time
Timeout: grace period < grace period	Time out: grace period - duration 1 cannot be killed	grace period

Terminating Pod的经验分享

terminationGracePeriodSeconds默认时长30秒

如果不关心Pod的终止时长，那么无需采取特殊措施

如果希望快速终止应用进程，那么可采取如下方案

- 在preStop script中主动退出进程
- 在主容器进程中使用特定的初始化进程

优雅的初始化进程应该

- 正确处理系统信号量，将信号量转发给子进程
- 在主进程退出之前，需要先等待并确保所有子进程退出
- 监控并清理孤儿子进程

在Kubernetes上部署应用的挑战

资源规划

- 每个实例需要多少计算资源
 - CPU/GPU?
 - Memory
- 超售需求
- 每个实例需要多少存储资源
 - 大小
 - 本地还是网盘
 - 读写性能
 - Disk IO
- 网络需求
 - 整个应用总体QPS和带宽

存储带来的挑战

多容器之间共享存储，最简方案是emptyDir

带来的挑战：

- emptyDir需要控制 size limit，否则无限扩张的应用会撑爆主机磁盘导致主机不可用，进而导致大规模集群故障
- emptyDir size limit生效以后，kubelet会定期对容器目录执行du操作，会导致些许的性能影响
- size limit达到以后，Pod会被驱逐，原Pod的日志配置等信息会消失

应用配置

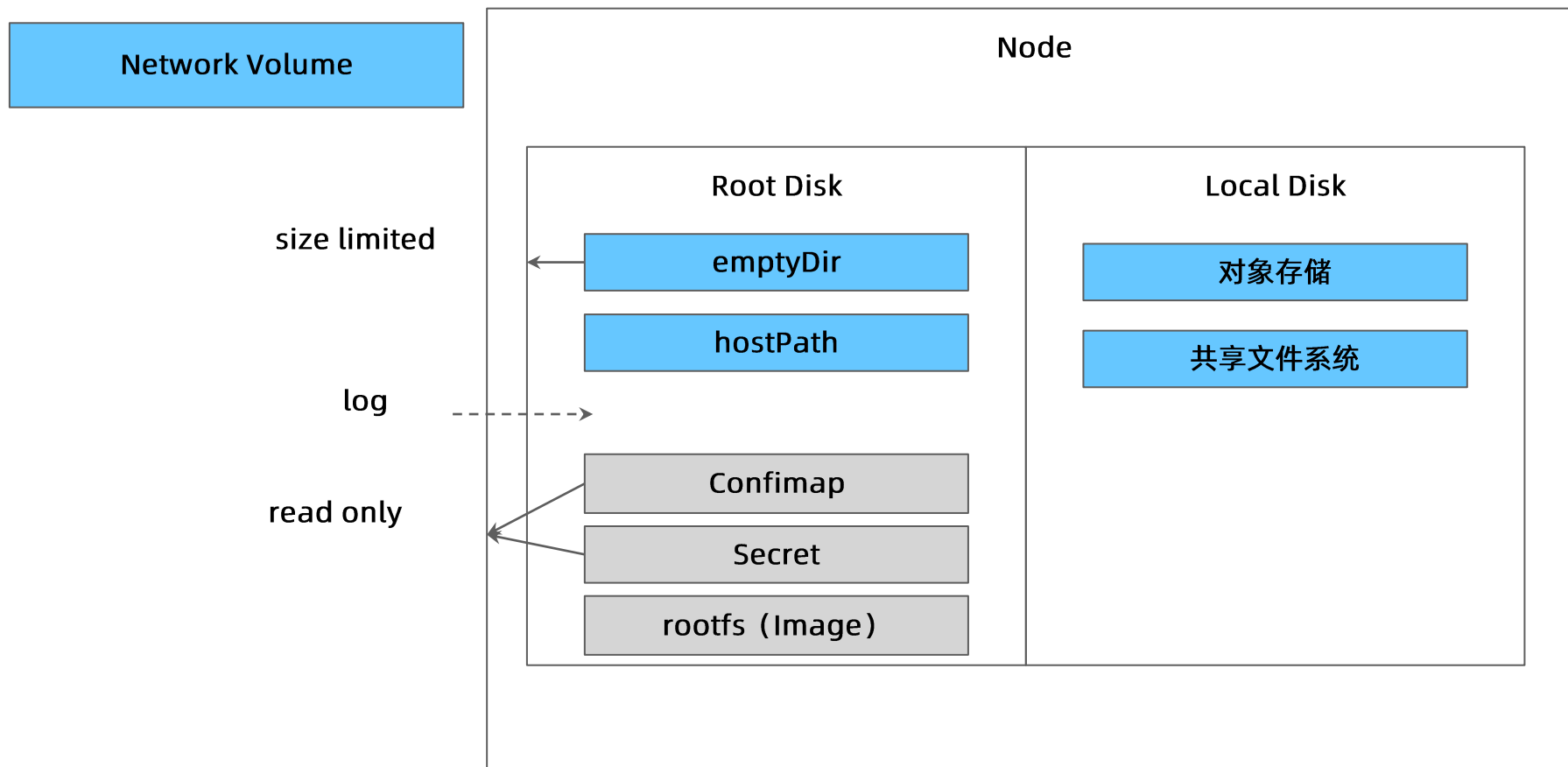
传入方式

- Environment Variables
- Volume Mount

数据来源

- ConfigMap
- Secret
- Downward API

数据应该如何保存



数据应该如何保存

存储卷类型	容器重启后是否存在	Pod重建后数据是否存在	是否有大小控制	注意
emptyDir	是	否	是	
hostPath	是	否	否	需要额外权限控制
Local volume	是	否	是	无备份
Network volume	是	是	是	
rootFS	否	否	否	不要写任何数据

注意控制日志写入速度，防止操作系统在配置日志滚动窗口期内把硬盘写满

容器应用可能面临的进程中断

类型	影响	建议
kubelet 升级	• 不重建容器	无影响
	• 重建容器 • Pod进程会被重启，服务	• 冗余部署 • 跨故障域部署
主机操作系统升级 节点手工重启	• 节点重启 • Pod进程会被终止数分钟（10分钟左右）	• 跨故障域部署 • 增加Liveness, Readiness探针 • 设置合理的NotReady node的Toleration时间
节点下架，送修	• 节点会drain，重启或者从集群中删除 • Pod进程会被终止数分钟	• 跨故障域部署 • 利用Pod disruption budget避免节点被drain导致Pod被意外删除而影响业务 • 利用preStop script 做数据备份等操作
	• Pods will be down for about 10 minutes	• 跨故障域部署 • 设置合理的NotReady node的Toleration时间
节点崩溃	• Pod进程会被终止15分钟左右	• 跨故障域部署

高可用部署方式

多少实例

更新策略

- maxSurge
- maxUnavailable（需要考虑ResourceQuota的限制）

深入理解PodTemplateHash导致的应用的易变性

课后作业（第一部分）

现在你对Kubernetes的控制面板的工作机制是否有了深入的了解呢？

是否对如何构建一个优雅的云上应用有了深刻的认识，那么接下来用最近学过的知识把你之前编写的http以优雅的方式部署起来吧，你可能需要审视之前代码是否能满足优雅上云的需求。

作业要求：编写Kubernetes 部署脚本将httpserver部署到kubernetes集群，以下是你可以思考的维度

- 优雅启动
- 优雅终止
- 资源需求和QoS保证
- 探活
- 日常运维需求，日志等级
- 配置和代码分离

2. 服务发现

服务发布

需要把服务发布至集群内部或者外部，服务的不同类型

- ClusterIP (Headless)
- NodePort
- LoadBalancer
- ExternalName

证书管理和七层负载均衡的需求

需要gRPC负载均衡如何做？

DNS需求

与上下游服务的关系

服务发布的挑战

kube-dns

- DNS TTL问题

Service

- ClusterIP只能对内
- kube-proxy支持的iptables/ipvs规模有限
- IPVS的性能和生产化问题
- kube-proxy的drift问题
- 频繁的Pod变动（spec change, failover, crashloop）导致LB频繁变更
- 对外发布的Service需要与企业ELB集成
- 不支持gRPC
- 不支持自定义DNS和高级路由功能

Ingress

- Spec的成熟度？

其他可选方案？

跨地域部署

需要多少实例

如何控制失败域，部署在几个地区，AZ，集群？

如何进行精细的流量控制

如何做按地域的顺序更新

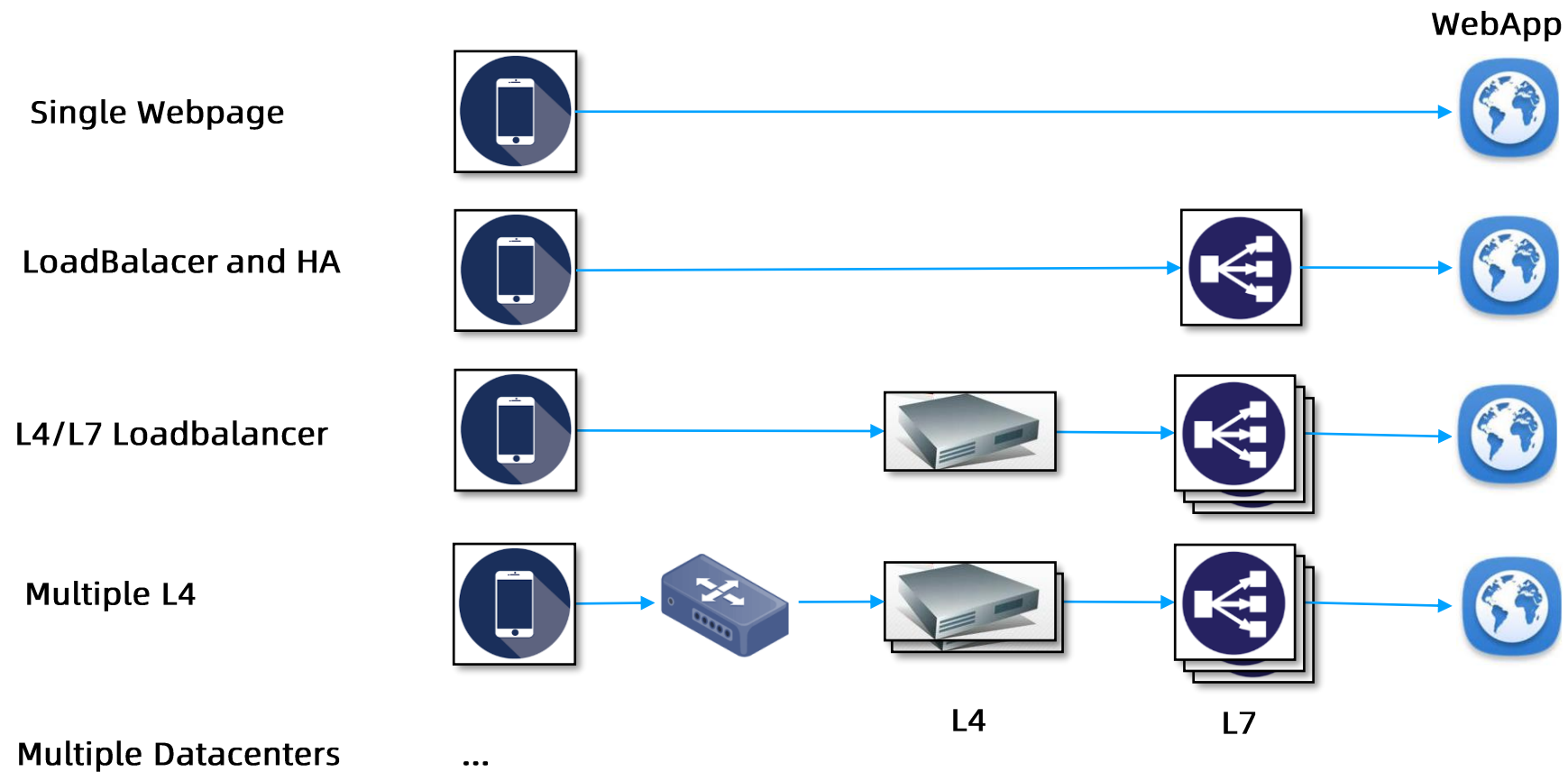
如何回滚

3. 微服务架构下的高可用挑战

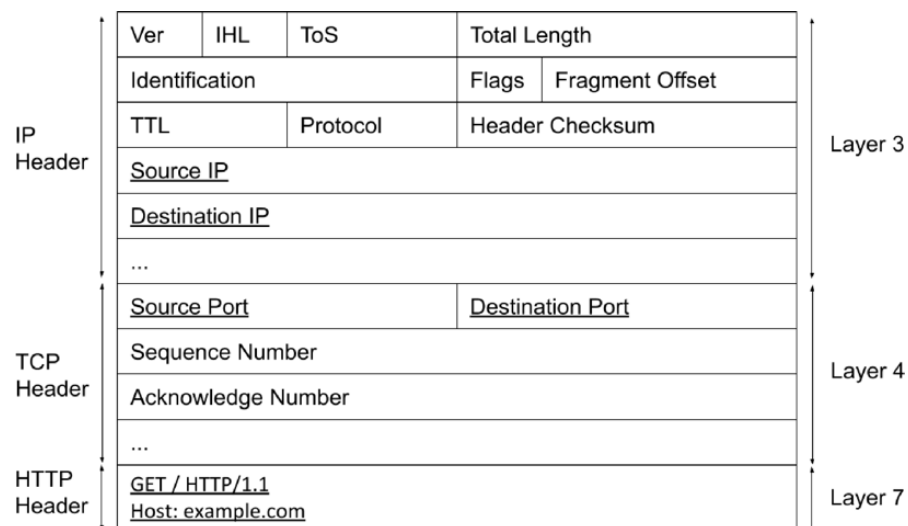
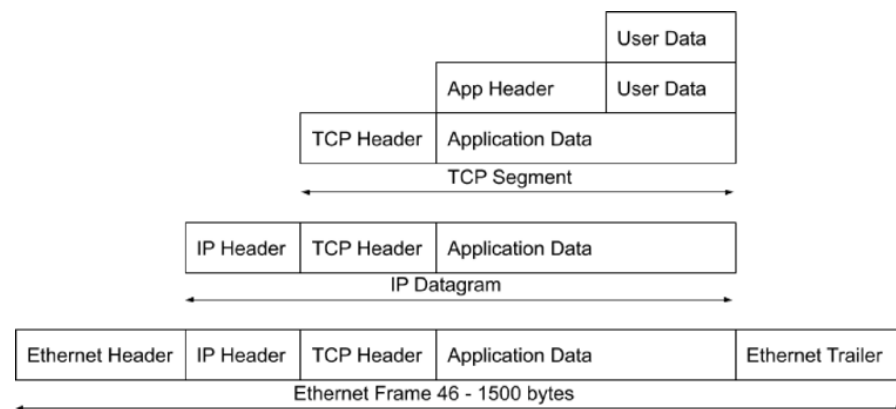
服务发现

- 微服务架构是由一系列职责单一的细粒度服务构成的分布式网状结构，服务之间通过轻量机制进行通信，这时候必然引入一个服务注册发现问题，也就是说服务提供方要注册通告服务地址，服务的调用方要能发现目标服务。
- 同时服务提供方一般以集群方式提供服务，也就引入了负载均衡和健康检查问题。

互联网架构发展历程

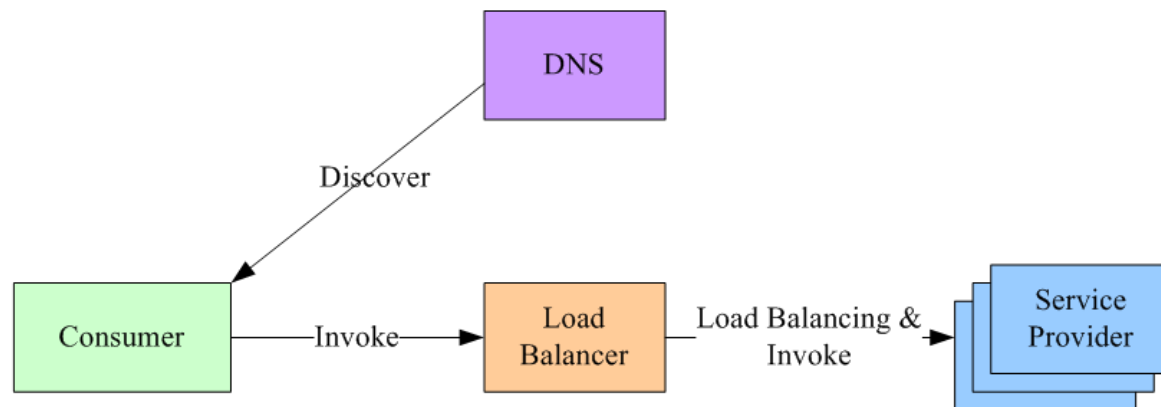


理解网络包格式



集中式LB服务发现

- 在服务消费者和服务提供者之间有一个独立的LB。
- LB上有所有服务的地址映射表，通常由运维配置注册。
- 当服务消费方调用某个目标服务时，它向LB发起请求，由LB以某种策略（比如Round-Robin）做负载均衡后将请求转发到目标服务。
- LB一般具备健康检查能力，能自动摘除不健康的服务实例。
- 服务消费方通过DNS发现LB，运维人员为服务配置一个DNS域名，这个域名指向LB。

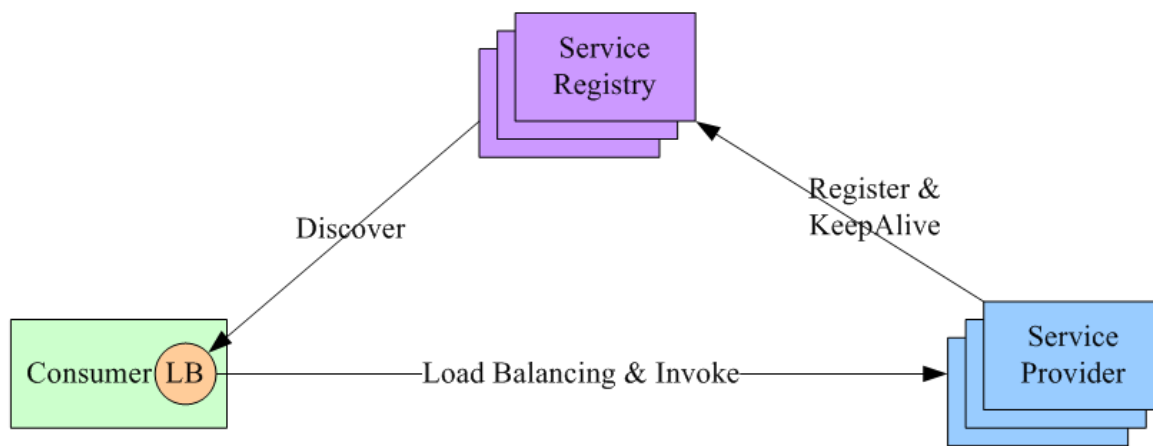


集中式LB服务发现

- 集中式LB方案实现简单，在LB上也容易做集中式的访问控制，这一方案目前还是业界主流。
- 集中式LB的主要问题是单点问题，所有服务调用流量都经过LB，当服务数量和调用量大的时候，LB容易成为瓶颈，且一旦LB发生故障对整个系统的影响是灾难性的。
- LB在服务消费方和服务提供方之间增加了一跳(hop)，有一定性能开销。

进程内LB服务发现

- 进程内LB方案将LB的功能以库的形式集成到服务消费方进程里头，该方案也被称为客户端负载方案。
- 服务注册表(Service Registry)配合支持服务自注册和自发现，服务提供方启动时，首先将服务地址注册到服务注册表（同时定期报心跳到服务注册表以表明服务的存活状态）。
- 服务消费方要访问某个服务时，它通过内置的LB组件向服务注册表查询（同时缓存并定期刷新）目标服务地址列表，然后以某种负载均衡策略选择一个目标服务地址，最后向目标服务发起请求。
- 这一方案对服务注册表的可用性(Availability)要求很高，一般采用能满足高可用分布式一致的组件（例如 ZooKeeper, Consul, etcd等）来实现。

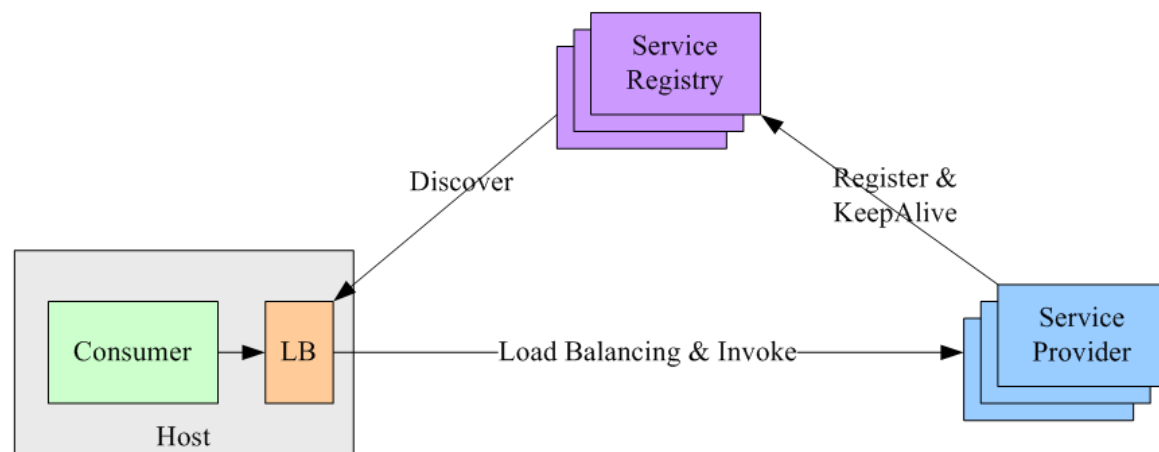


进程内LB服务发现

- 进程内LB是一种分布式模式，LB和服务发现能力被分散到每一个服务消费者的进程内部，同时服务消费方和服务提供方之间是直接调用，没有额外开销，性能比较好。该方案以客户库(Client Library)的方式集成到服务调用方进程里头，如果企业内有多种不同的语言栈，就要配合开发多种不同的客户端，有一定的研发和维护成本。
- 一旦客户端跟随服务调用方发布到生产环境中，后续如果要对客户库进行升级，势必要求服务调用方修改代码并重新发布，所以该方案的升级推广有不小的阻力。

独立LB进程服务发现

- 针对进程内LB模式的不足而提出的一种折中方案，原理和第二种方案基本类似。
- 不同之处是，将LB和服务发现功能从进程内移出来，变成主机上的一个独立进程，主机上的一个或者多个服务要访问目标服务时，他们都通过同一主机上的独立LB进程做服务发现和负载均衡。
- LB独立进程可以进一步与服务消费方进行解耦，以独立集群的形式提供高可用的负载均衡服务。
- 这种模式可以称之为真正的“软负载（Soft Load Balancing）”。



独立LB进程服务发现

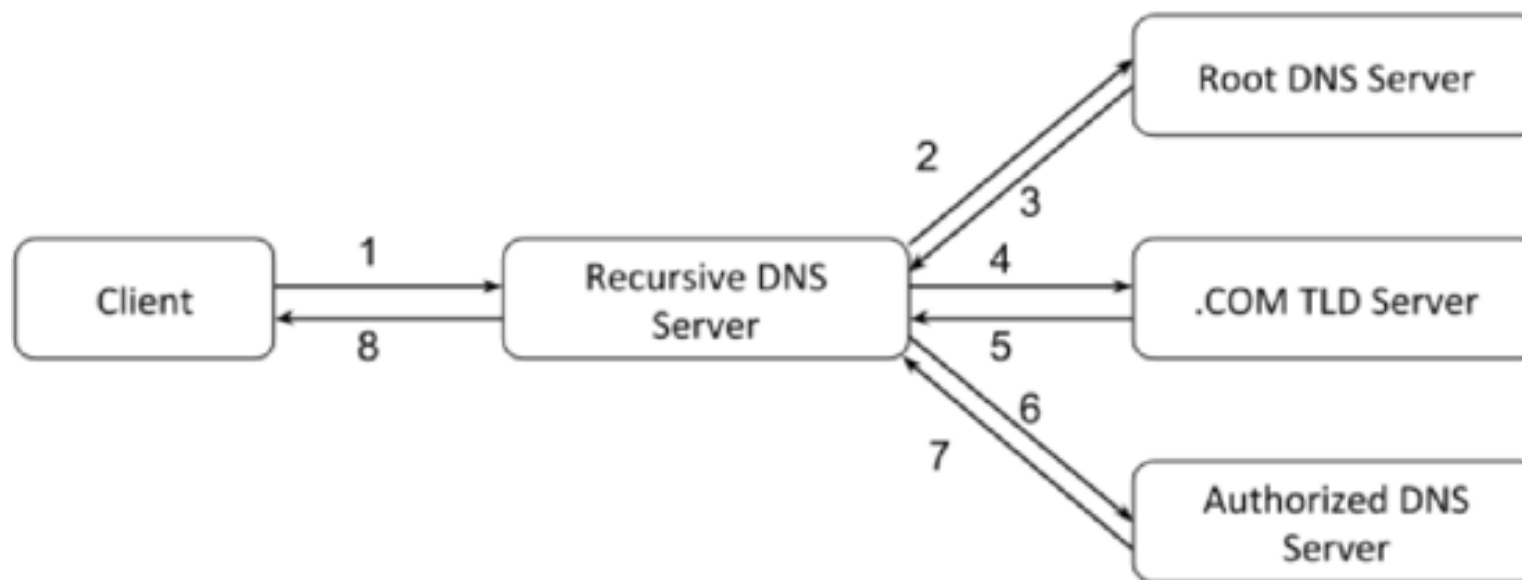
- 独立LB进程也是一种分布式方案，没有单点问题，一个LB进程挂了只影响该主机上的服务调用方。
- 服务调用方和LB之间是进程间调用，性能好。
- 简化了服务调用方，不需要为不同语言开发客户库，LB的升级不需要服务调用方改代码。
- 不足是部署较复杂，环节多，出错调试排查问题不方便。

负载均衡

- 系统的扩展可分为纵向（垂直）扩展和横向（水平）扩展。
 - 纵向扩展，是从单机的角度通过增加硬件处理能力，比如CPU处理能力，内存容量，磁盘等方面，实现服务器处理能力的提升，不能满足大型分布式系统（网站），大流量，高并发，海量数据的问题；
 - 横向扩展，通过添加机器来满足大型网站服务的处理能力。比如：一台机器不能满足，则增加两台或者多台机器，共同承担访问压力，这就是典型的集群和负载均衡架构。
- 负载均衡的作用（解决的问题）：
 - 解决并发压力，提高应用处理性能，增加吞吐量，加强网络处理能力；
 - 提供故障转移，实现高可用；
 - 通过添加或减少服务器数量，提供网站伸缩性，扩展性；
 - 安全防护，负载均衡设备上做一些过滤，黑白名单等处理。

DNS负载均衡

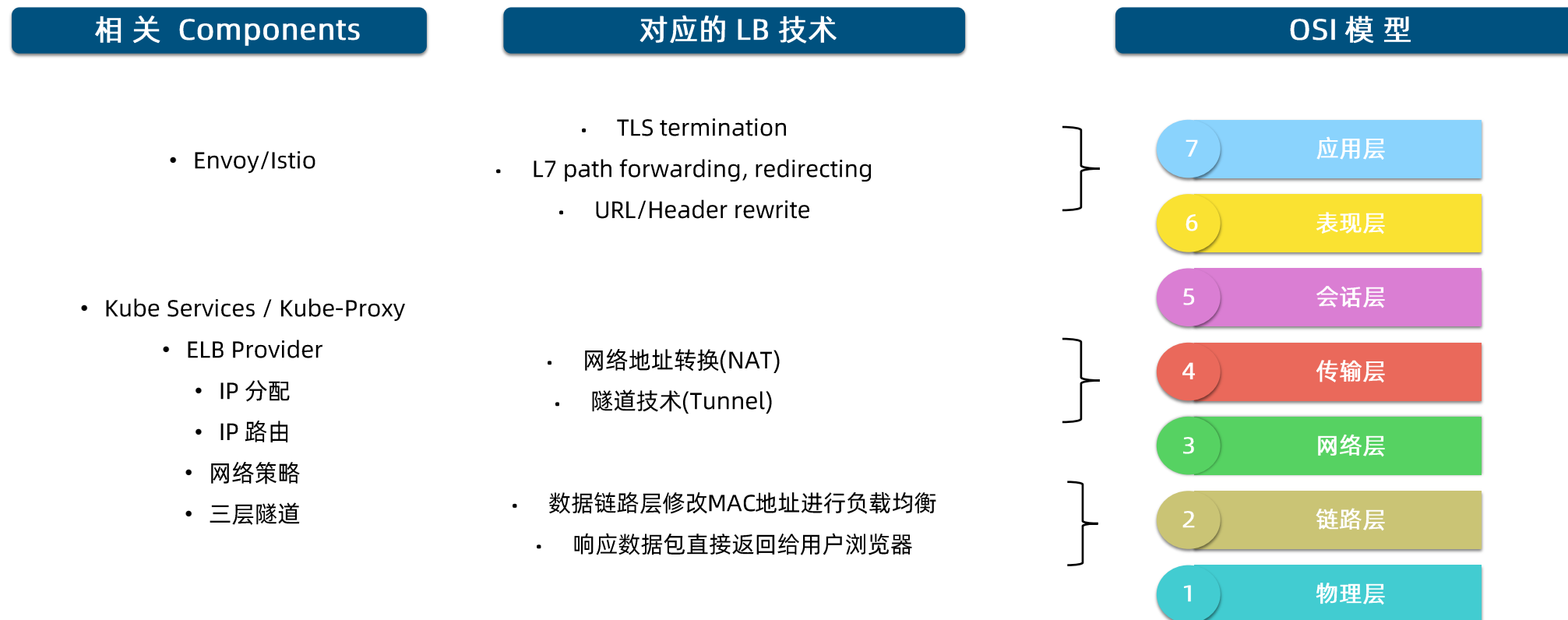
最早的负载均衡技术，利用域名解析实现负载均衡，在DNS服务器，配置多个A记录，这些A记录对应的服务器构成集群。



DNS负载均衡

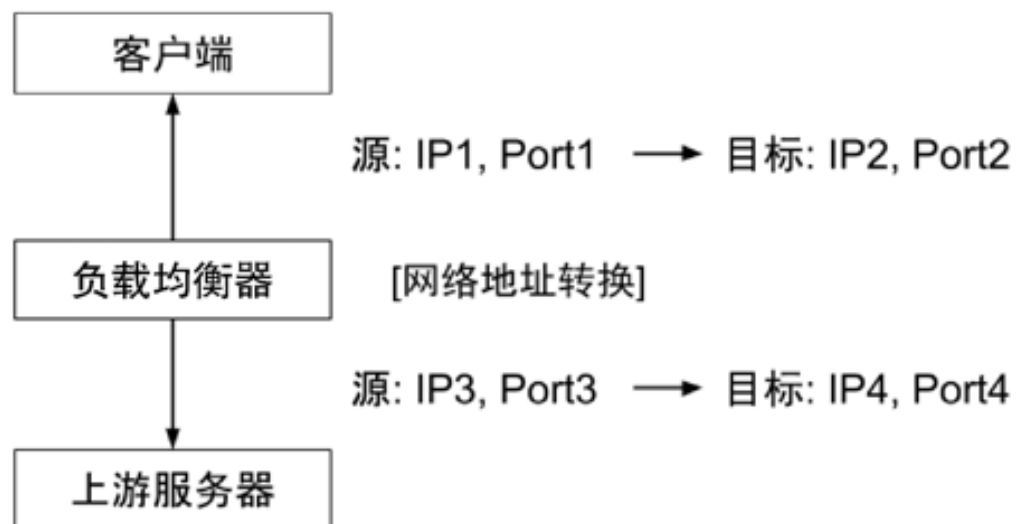
优点	缺点
• 使用简单：负载均衡工作，交给DNS服务器处理，省掉了负载均衡服务器维护的麻烦； • 提高性能：可以支持基于地址的域名解析，解析成距离用户最近的服务器地址，可以加快访问速度，改善性能。	• 可用性差：DNS解析是多级解析，新增/修改DNS后，解析时间较长；解析过程中，用户访问网站将失败； • 扩展性低：DNS负载均衡的控制权在域名商那里，无法对其做更多的改善和扩展； • 维护性差：也不能反映服务器的当前运行状态；支持的算法少；不能区分服务器的差异。（不能根据系统与服务的状态来判断负载）

负载均衡技术概览



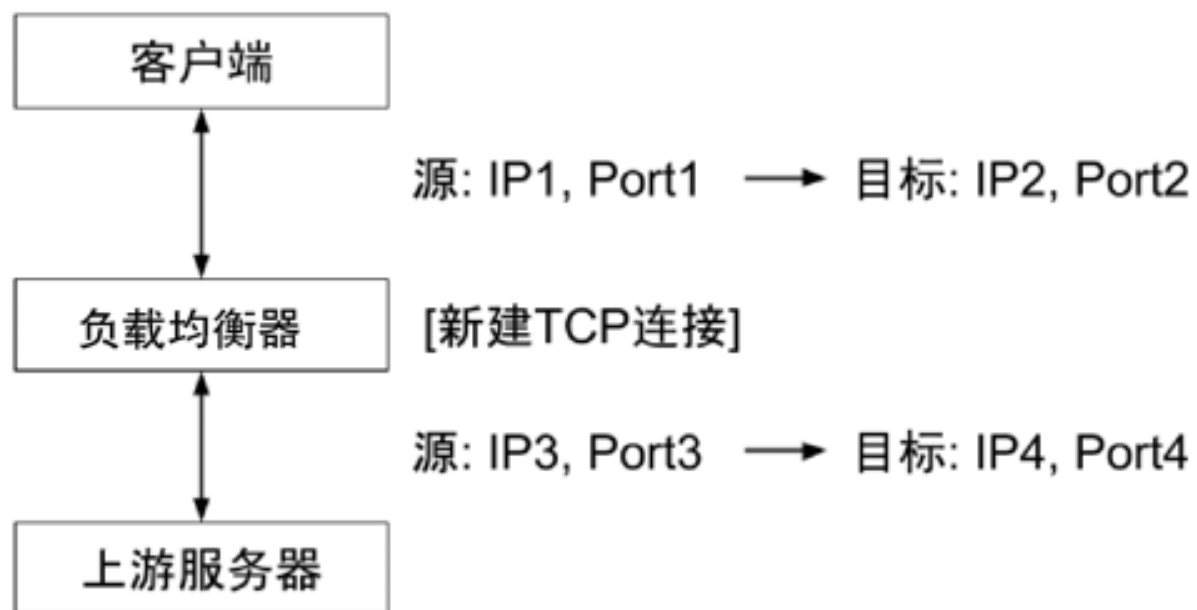
网络地址转换

网络地址转换（Network Address Translation, NAT）通常通过修改数据包的源地址（Source NAT）或目标地址（Destination NAT）来控制数据包的转发行为



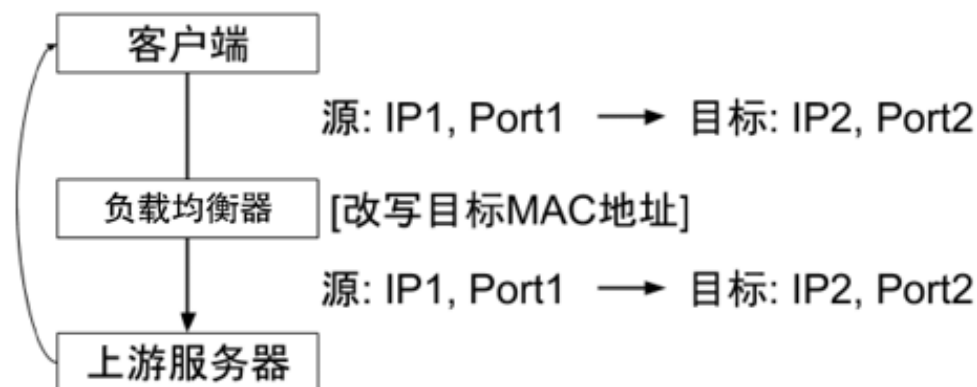
新建TCP连接

为记录原始客户端IP地址，负载均衡功能不仅需要进行数据包的源目标地址修改，同时要记录原始客户端IP地址，基于简单的NAT无法满足此需求，于是衍生出了基于传输层协议的负载均衡的另一种方案——TCP/UDP Termination方案



链路层负载均衡

- 在通信协议的数据链路层修改MAC地址进行负载均衡。
- 数据分发时，不修改IP地址，只修改目标MAC地址，配置真实物理服务器集群所有机器虚拟IP和负载均衡服务器IP地址一致，达到不修改数据包的源地址和目标地址，进行数据分发的目的。
- 实际处理服务器IP和数据请求目的IP一致，不需要经过负载均衡服务器进行地址转换，可将响应数据包直接返回给用户浏览器，避免负载均衡服务器网卡带宽成为瓶颈。也称为直接路由模式（DR模式）。



隧道技术

负载均衡中常用的隧道技术是IP over IP，其原理是保持原始数据包IP头不变，在IP头外层增加额外的IP包头后转发给上游服务器。

上游服务器接收IP数据包，解开外层IP包头后，剩下的是原始数据包。

同样的，原始数据包中的目标IP地址要配置在上游服务器中，上游服务器处理完数据请求以后，响应包通过网关直接返回给客户端。

4. Service对象

Service对象

- Service Selector
 - Kubernetes允许将Pod对象通过标签（Label）进行标记，并通过Service Selector定义基于Pod标签的过滤规则，以便选择服务的上游应用实例
- Ports
 - Ports属性中定义了服务的端口、协议目标端口等信息

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-service
spec:
  selector:
    app: nginx
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
```


Endpoint对象

- 当Service的selector不为空时，Kubernetes Endpoint Controller会侦听服务创建事件，创建与Service同名的Endpoint对象
- selector能够选取的所有PodIP都会被配置到addresses属性中
 - 如果此时selector所对应的filter查询不到对应的Pod，则addresses列表为空
 - 默认配置下，如果此时对应的Pod为not ready状态，则对应的PodIP只会出现在subsets的notReadyAddresses属性中，这意味着对应的Pod还没准备好提供服务，不能作为流量转发的目标。
 - 如果设置了PublishNotReadyAddress为true，则无论Pod是否就绪都会被加入readyAddress list

```
apiVersion: v1
kind: Endpoint
metadata:
  name: nginx-service
subsets:
- addresses:
  - ip: 10.1.1.21
    nodeName: minikube
    targetRef:
      kind: Pod
      name: nginx-deployment-5754944d6c-
hnw27
    namespace: default
    resourceVersion: "722191"
    uid: 8a3390ae-2f8e-47bf-b8dd-
70fae7fb0d32
```

EndpointSlice对象

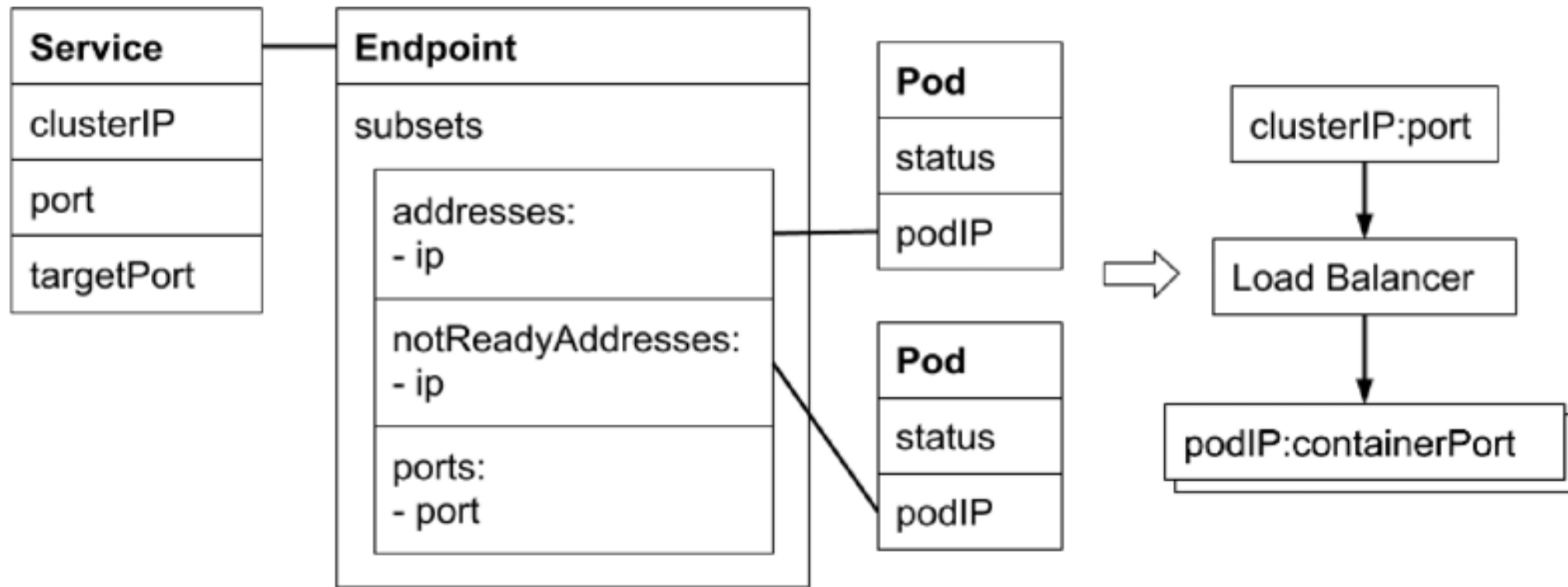
- 当某个Service对应的backend Pod较多时, Endpoint对象就会因保存的地址信息过多而变得异常庞大
- Pod状态的变更会引起Endpoint的变更, Endpoint的变更会被推送至所有节点, 从而导致持续占用大量网络带宽
- EndpointSlice对象, 用于对Pod较多的Endpoint进行切片, 切片大小可以自定义

```
apiVersion: discovery.k8s.io/v1beta1
kind: EndpointSlice
metadata:
  name: example-abc
  labels:
    kubernetes.io/service-name: example
addressType: IPv4
ports:
  - name: http
    protocol: TCP
    port: 80
endpoints:
  - addresses:
    - "10.1.2.3"
    conditions:
      ready: true
    hostname: pod-1
    topology:
      kubernetes.io/hostname: node-1
      topology.kubernetes.io/zone: us-west2-a
```

不定义Selector的Service

- 用户创建了Service但不定义Selector
 - Endpoint Controller不会为该Service自动创建Endpoint
 - 用户可以手动创建Endpoint对象，并设置任意IP地址到Address属性
 - 访问该服务的请求会被转发至目标地址
- 通过该类型服务，可以为集群外的一组Endpoint创建服务

Service、Endpoint和Pod的对应关系



Service类型

- clusterIP
 - Service的默认类型，服务被发布至仅集群内部可见的虚拟IP地址上。
 - 在API Server启动时，需要通过service-cluster-ip-range参数配置虚拟IP地址段，API Server中用于分配IP地址和端口的组件，当该组件捕获Service对象并创建事件时，会从配置的虚拟IP地址段中取一个有效的IP地址，分配给该Service对象。
- nodePort
 - 在API Server启动时，需要通过node-port-range参数配置nodePort的范围，同样的，API Server组件会捕获Service对象并创建事件，即从配置好的nodePort范围取一个有效端口，分配给该Service。
 - 每个节点的kube-proxy会尝试在服务分配的nodePort上建立侦听器接收请求，并转发给服务对应的后端Pod实例。
- LoadBalancer
 - 企业数据中心一般会采购一些负载均衡器，作为外网请求进入数据中心内部的统一流量入口。
 - 针对不同的基础架构云平台，Kubernertes Cloud Manager提供支持不同供应商API的Service Controller。如果需要在Openstack云平台上搭建Kubernetes集群，那么只需提供一份openstack.rc，Openstack Service Controller即可通过调用LBaaS API完成负载均衡配置。

其他类型服务

- Headless Service

Headless服务是用户将clusterIP显示定义为None的服务。

无头的服务意味着Kubernetes不会为该服务分配统一入口，包括clusterIP，nodePort等

- ExternalName Service

为一个服务创建别名

Headless和externalName服务的使用场景？我们随后讨论

Service Topology

- 一个网络调用的延迟受客户端和服务端所处位置的影响，两者是否在同一节点、同一机架、同一可用区、同一数据中心，都会影响参与数据传输的设备数量
- 在分布式系统中，为保证系统的高可用，往往需要控制应用的错误域（Failure Domain），比如通过反亲和性配置，将一个应用的多个副本部署在不同机架，甚至不同的数据中心
- Kubernetes提供通用标签来标记节点所处的物理位置，如：

`topology.kubernetes.io/zone: us-west2-a`

`failure-domain.beta.kubernetes.io/region: us-west`

`failure-domain.tess.io/network-device: us-west05-ra053`

`failure-domain.tess.io/rack: us_west02_02-314_19_12`

`kubernetes.io/hostname: node-1`

Service Topology

- Service引入了topologyKeys属性，可以通过如下设置来控制流量
 - 当topologyKeys设置为["kubernetes.io/hostname"]时，调用服务的客户端所在节点上如果有服务实例正在运行，则该实例处理请求，否则，调用失败。
 - 当topologyKeys设置为["kubernetes.io/hostname", "topology.kubernetes.io/zone", "topology.kubernetes.io/region"]时，若同一节点有对应的服务实例，则请求会优先转发至该实例。否则，顺序查找当前zone及当前region是否有服务实例，并将请求按顺序转发。
 - 当topologyKeys设置为["topology.kubernetes.io/ zone", "*"]时，请求会被优先转发至当前zone的服务实例。如果当前zone不存在服务实例，则请求会被转发至任意服务实例。

5. kube-proxy

kube-proxy

每台机器上都运行一个kube-proxy服务，它监听API server中service和endpoint的变化情况，并通过iptables等来为服务配置负载均衡（仅支持TCP和UDP）。

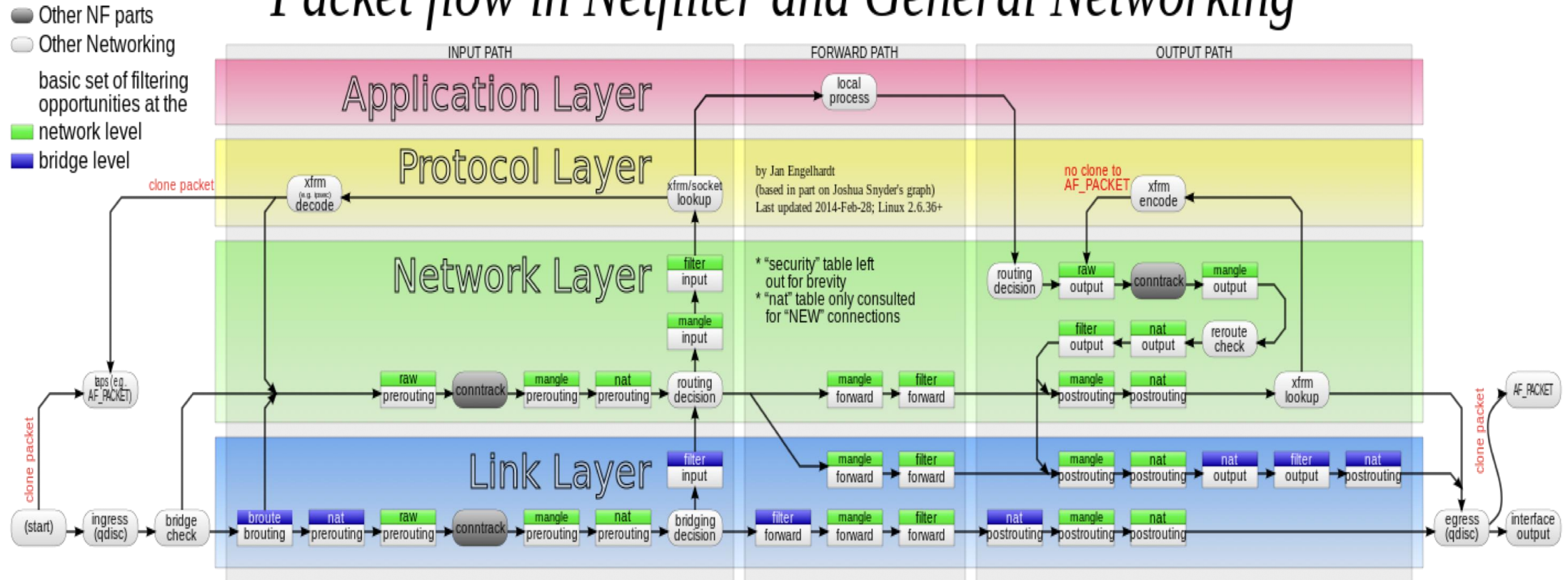
kube-proxy可以直接运行在物理机上，也可以以static pod或者DaemonSet的方式运行。

kube-proxy当前支持以下几种实现

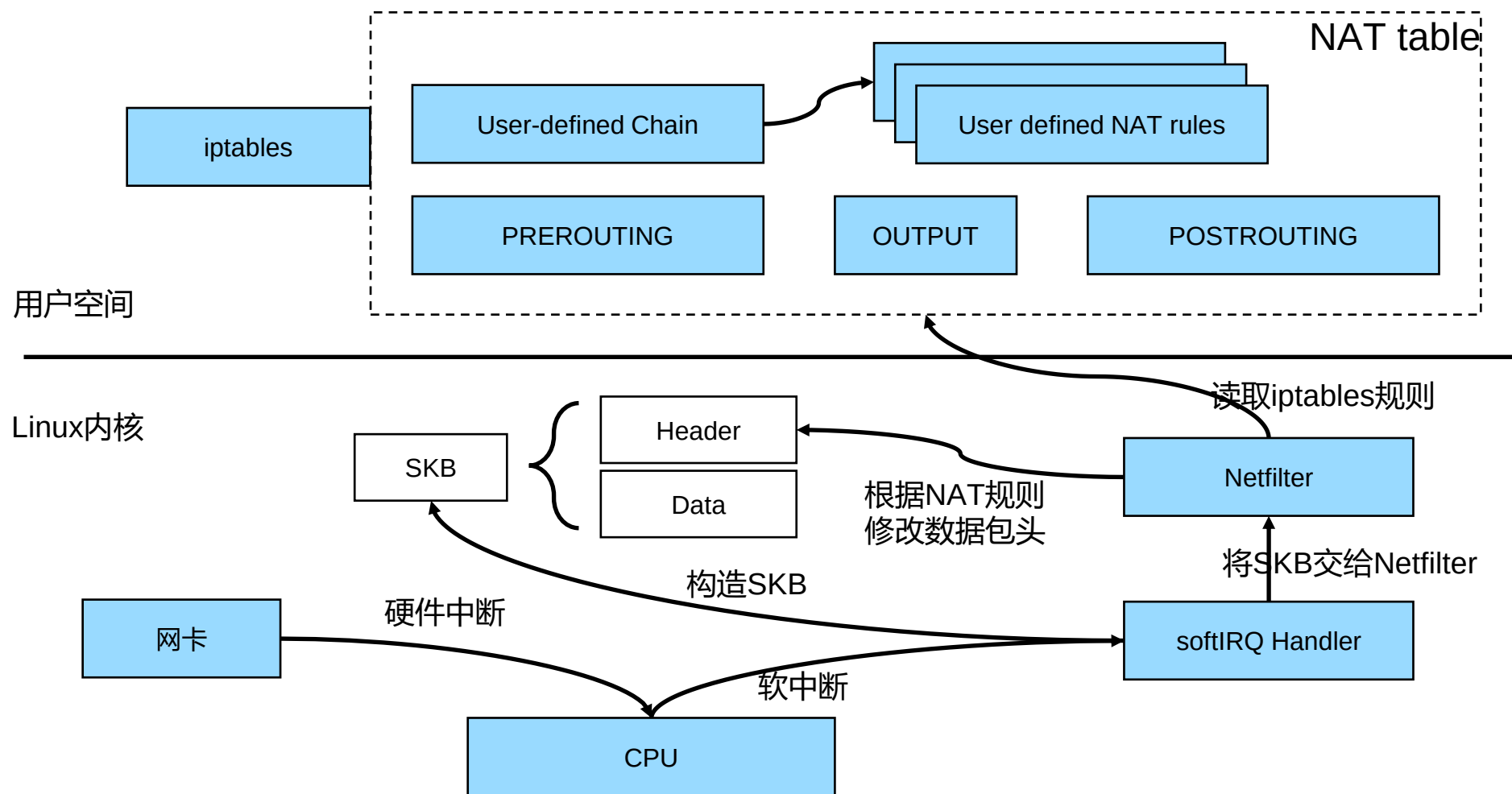
- userspace：最早的负载均衡方案，它在用户空间监听一个端口，所有服务通过iptables转发到这个端口，然后在其内部负载均衡到实际的Pod。该方式最主要的问题是效率低，有明显的性能瓶颈。
- iptables：目前推荐的方案，完全以iptables规则的方式来实现service负载均衡。该方式最主要的问题是在服务多的时候产生太多的iptables规则，非增量式更新会引入一定的时延，大规模情况下有明显的性能问题
- ipvs：为解决iptables模式的性能问题，v1.8新增了ipvs模式，采用增量式更新，并可以保证service更新期间连接保持不断开
- winuserspace：同userspace，但仅工作在windows上

Linux内核处理数据包：Netfilter框架

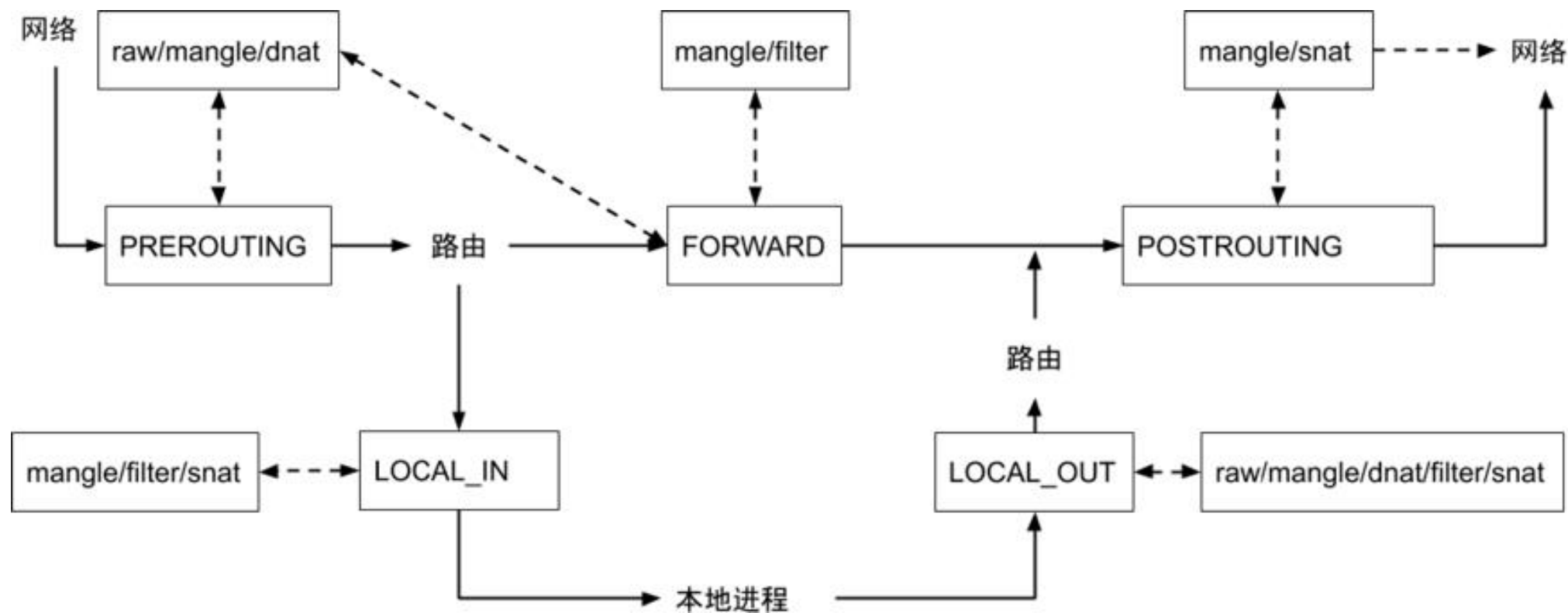
Packet flow in Netfilter and General Networking



Netfilter和iptables



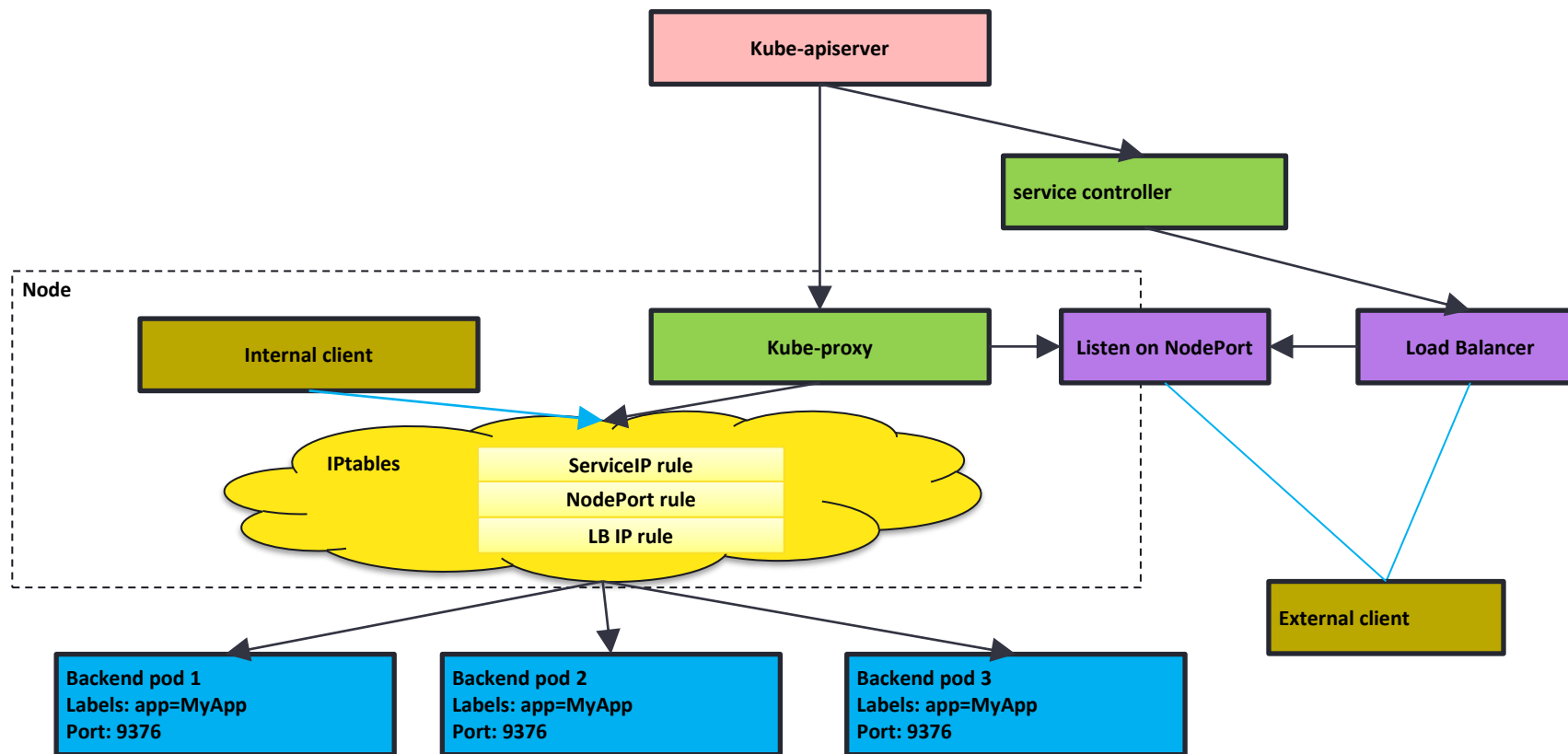
iptables



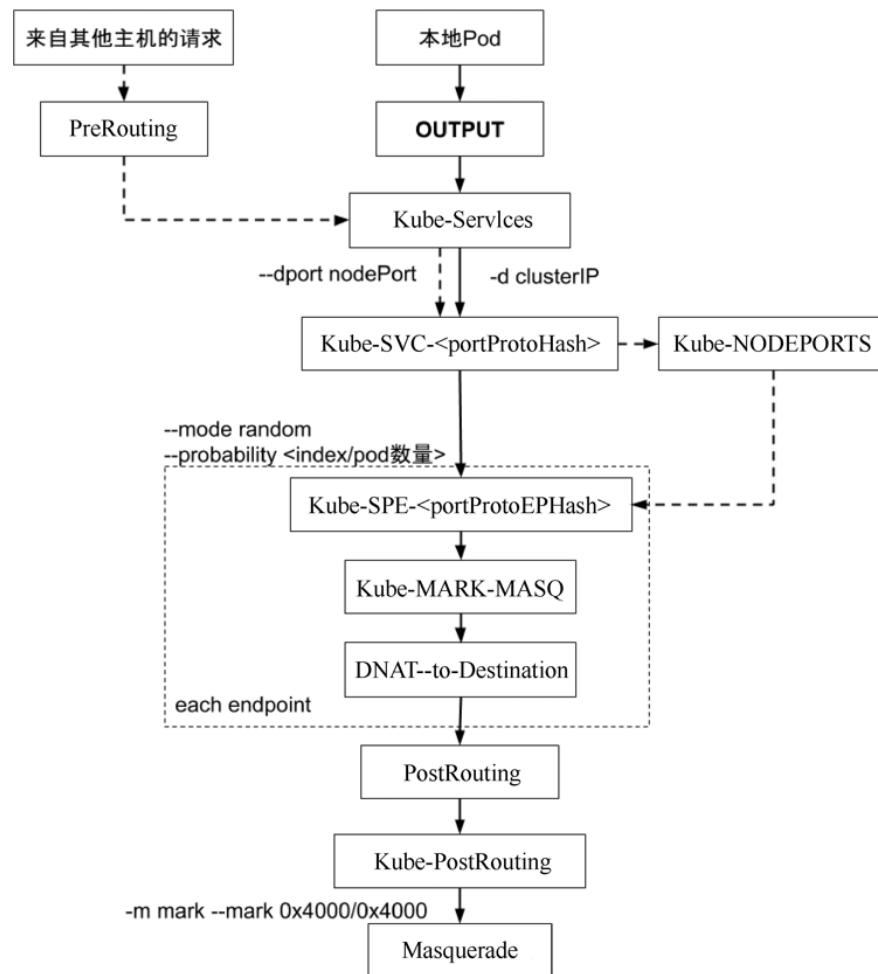
iptables支持的锚点

table/chain	PREROUTING	INPUT	FORWARD	OUTPUT	POSTROUTING
raw	支持			支持	
mangle	支持	支持	支持	支持	支持
dnat	支持			支持	
filter		支持	支持	支持	
snat		支持		支持	支持

kube-proxy工作原理



Kubernetes iptables规则



iptables示例

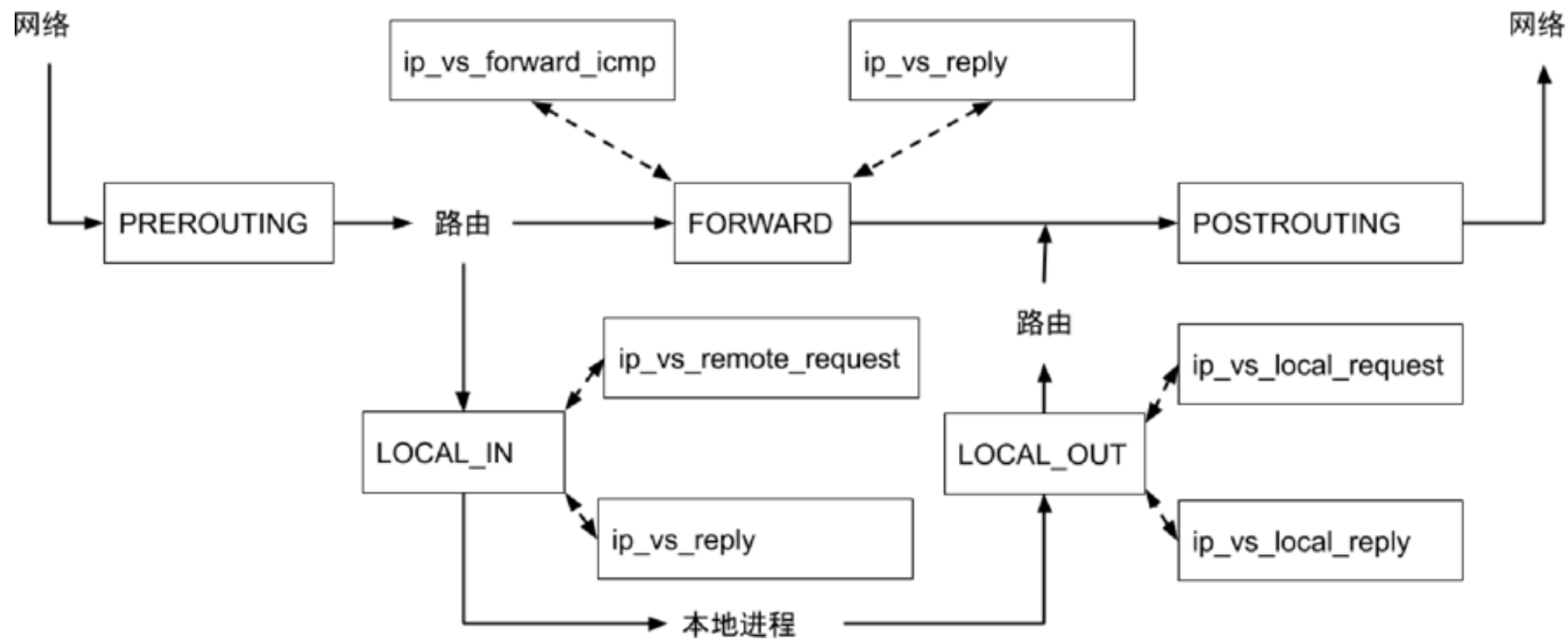
```
-A KUBE-MARK-DROP -j MARK --set-xmark 0x8000/0x8000
-A KUBE-MARK-MASQ -j MARK --set-xmark 0x4000/0x4000
-A KUBE-POSTROUTING -m comment --comment "kubernetes service traffic requiring SNAT" -m mark --mark 0x4000/0x4000 -j MASQUERADE

-A KUBE-SEP-55QZ6T7MF3AHPOOB -s 10.244.1.6/32 -m comment --comment "default/http:" -j KUBE-MARK-MASQ
-A KUBE-SEP-55QZ6T7MF3AHPOOB -p tcp -m comment --comment "default/http:" -m tcp -j DNAT --to-destination 10.244.1.6:80

-A KUBE-SEP-KJZJRL2KRWMXNR3J -s 10.244.1.5/32 -m comment --comment "default/http:" -j KUBE-MARK-MASQ
-A KUBE-SEP-KJZJRL2KRWMXNR3J -p tcp -m comment --comment "default/http:" -m tcp -j DNAT --to-destination 10.244.1.5:80

-A KUBE-SERVICES -d 10.101.85.234/32 -p tcp -m comment --comment "default/http: cluster IP" -m tcp --dport 80 -j KUBE-SVC-7IMAZDGB2ONQNK4Z
-A KUBE-SVC-7IMAZDGB2ONQNK4Z -m comment --comment "default/http:" -m statistic --mode random --probability 0.500000000000 -j KUBE-SEP-KJZJRL2KRWMXNR3J
-A KUBE-SVC-7IMAZDGB2ONQNK4Z -m comment --comment "default/http:" -j KUBE-SEP-55QZ6T7MF3AHPOOB
```

IPVS



IPVS支持的锚点和核心函数

HOOK	函 数	核心函数	Priority
NF_INET_LOCAL_IN	ip_vs_reply4	ip_vs_out	NF_IP_PRI_NAT_SRC - 2
NF_INET_LOCAL_IN	ip_vs_remote_request4	ip_vs_in	NF_IP_PRI_NAT_SRC - 1
NF_INET_LOCAL_OUT	ip_vs_local_reply4	ip_vs_out	NF_IP_PRI_NAT_DST + 1
NF_INET_LOCAL_OUT	ip_vs_local_request4	ip_vs_in	NF_IP_PRI_NAT_DST + 2
NF_INET_FORWARD	ip_vs_forward_icmp	ip_vs_in_icmp	99
NF_INET_FORWARD	ip_vs_reply4	ip_vs_out	100

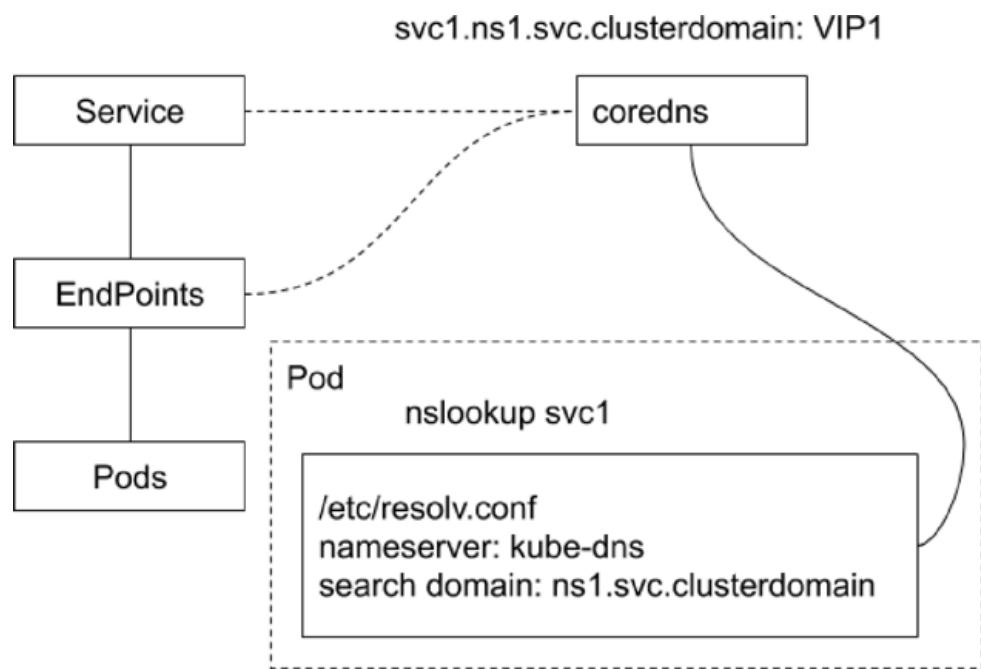
域名服务

- Kubernetes Service通过虚拟IP地址或者节点端口为用户应用提供访问入口
- 然而这些IP地址和端口是动态分配的，如果用户重建一个服务，其分配的clusterIP和nodePort，以及LoadBalancerIP都是会变化的，我们无法把一个可变的入口发布出去供他人访问
- Kubernetes提供了内置的域名服务，用户定义的服务会自动获得域名，而无论服务重建多少次，只要服务名不改变，其对应的域名就不会改变

CoreDNS

CoreDNS包含一个内存态DNS，以及与其他controller类似的控制器

CoreDNS的实现原理是，控制器监听Service和EndPoint的变化并配置DNS，客户端Pod在进行域名解析时，从 CoreDNS 中查询服务对应的地址记录



不同类型服务的DNS记录

- 普通Service
 - ClusterIP、nodePort、LoadBalancer 类型的Service都拥有API Server分配的ClusterIP, CoreDNS会为这些Service创建FQDN格式为 `$svcname.$namespace.svc.$clusterdomain`: clusterIP的A记录及PTR记录, 并为端口创建SRV记录。
- Headless Service
 - 顾名思义, 无头, 是用户在Spec显式指定ClusterIP为None的Service, 对于这类Service, API Server不会为其分配ClusterIP。CoreDNS为此类Service创建多条A记录, 并且目标为每个就绪的PodIP。
 - 另外, 每个Pod会拥有一个FQDN格式为 `$podname.$svcname.$namespace.svc.$clusterdomain`的A记录指向PodIP。
- ExternalName Service
 - 此类Service用来引用一个已经存在的域名, CoreDNS会为该Service创建一个CName记录指向目标域名。

Kubernetes中的域名解析

- Kubernetes Pod有一个与DNS策略相关的属性DNSPolicy，默认值是ClusterFirst
- Pod启动后的/etc/resolv.conf会被改写，所有的地址解析优先发送至CoreDNS

```
$ cat /etc/resolv.conf
```

```
search ns1.svc.cluster.local svc.cluster.local cluster.local
```

```
nameserver 192.168.0.10
```

```
options ndots:4
```

- 当Pod启动时，同一Namespace的所有Service都会以环境变量的形式设置到容器内
- 影响？

关于DNS的落地实践

Kubernetes作为企业基础架构的一部分，Kubernetes服务也需要发布到企业DNS，需要定制企业DNS控制器

- 对于Kubernetes中的服务，在企业DNS同样创建A/PTR/SRV records（通常解析地址是LoadBalancer VIP）
- 针对headless service，在PodIP可全局路由的前提下，按需创建DNS records
- Headless service的DNS记录，应该按需创建，否则对企业DNS冲击过大

服务在集群内通过CoreDNS寻址，在集群外通过企业DNS寻址，服务在集群内外有统一标识。

Kubernetes中的负载均衡技术

基于 L4 的服务

- 基于iptables/ipvs的分布式四层负载均衡技术
- 多种Load Balancer Provider提供与企业现有ELB的整合
- kube-proxy基于iptables rules为Kubernetes形成全局统一的distributed load balancer
- kube-proxy是一种mesh, Internal Client无论通过podip, nodeport还是LB VIP都经由kube-proxy跳转至pod
- 属于Kubernetes core

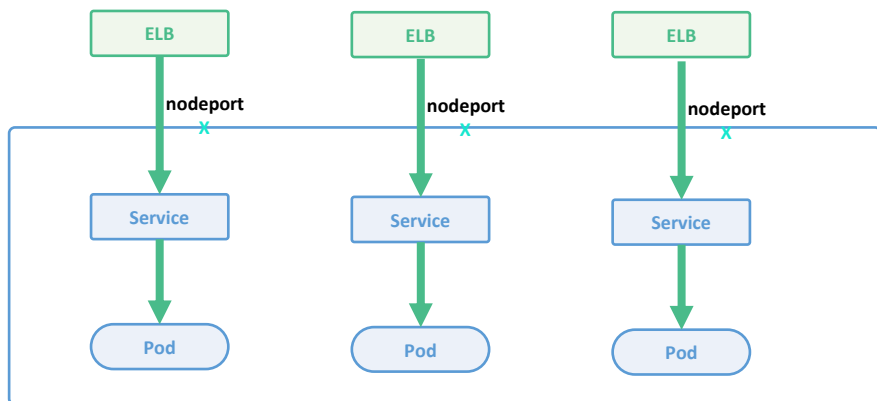
基于L7的Ingress

- 基于七层应用层，提供更多功能
- TLS termination
- L7 path forwarding
- URL/http header rewrite
- 与采用7层软件紧密相关

Service中的Ingress的对比

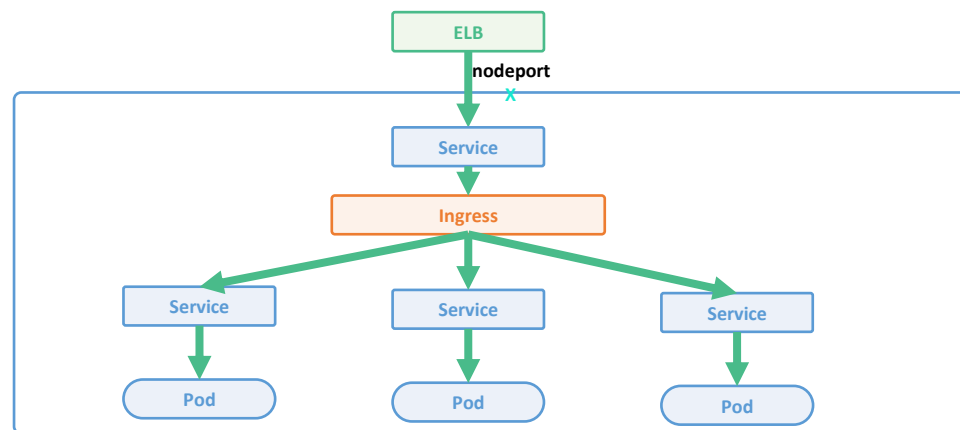
基于L4的服务

- 每个应用独占ELB, 浪费资源
- 为每个服务动态创建DNS记录, 频繁的DNS更新
- 支持TCP和UDP, 业务部门需要启动HTTPS服务, 自己管理证书



基于L7的Ingress

- 多个应用共享ELB, 节省资源
- 多个应用共享一个Domain, 可采用静态DNS配置
- TLS termination发生在Ingress层, 可集中管理证书
- 更多复杂性, 更多的网络hop



Ingress

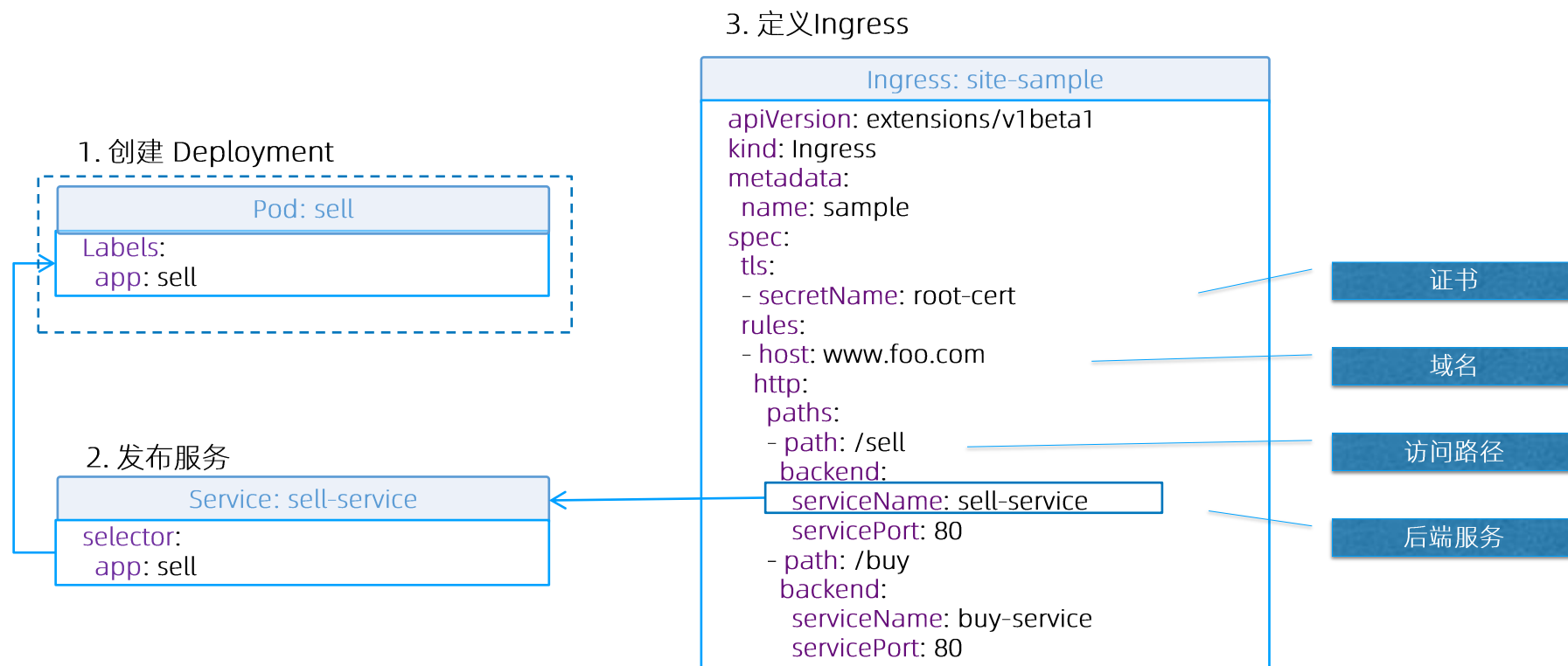
- Ingress

- Ingress是一层代理
- 负责根据hostname和path将流量转发到不同的服务上，使得一个负载均衡器用于多个后台应用
- Kubernetes Ingress Spec是转发规则的集合

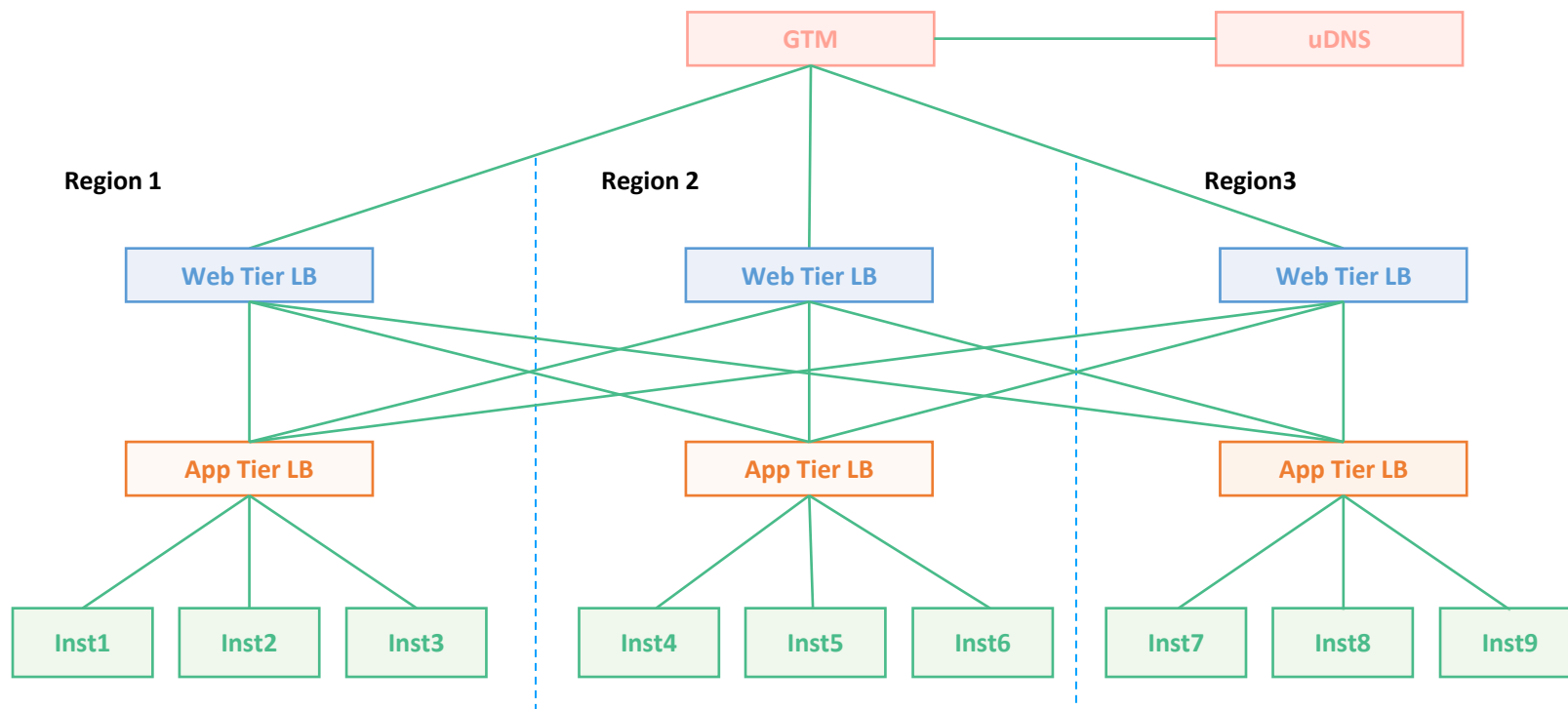
- Ingress Controller

- 确保实际状态（Actual）与期望状态（Desired）一致的Control Loop
- Ingress Controller确保
- 负载均衡配置
- 边缘路由配置
- DNS配置

Ingress



传统应用网络拓扑



三步构建Ingress Controller

- 复用Kubernetes LoadBalancer接口

```
GetLoadBalancer(clusterName string, service *api.Service) (*api.LoadBalancerStatus, bool, error)
EnsureLoadBalancer(clusterName string, service *api.Service, members []string) (*api.LoadBalancerStatus, error)
UpdateLoadBalancer(clusterName string, service *api.Service, members []string) (*api.LoadBalancerStatus, error)
EnsureLoadBalancerDeleted(clusterName string, service *api.Service) error
```

- 定义Informer, 监控ingress, secret, service, endpoint的变化, 加入相应的队列

```
lbc.ingLister.Store, lbc.ingController = framework.NewInformer(
    &cache.ListWatch{
        ListFunc: ingressListFunc(lbc.client, namespace),
        WatchFunc: ingressWatchFunc(lbc.client, namespace),
    },
    &extensions.Ingress{}, resyncPeriod, framework.ResourceEventHandlerFuncs{
        AddFunc: func(obj interface{}) {
            lbc.ingQueue.enqueueIng(obj)
        },
    },
)
```

- 启动worker, 遍历ingress队列

- ✓ 为Ingress Domain创建LoadBalancer service, 依赖service controller创建ingress vip
- ✓ 为Ingress Domain创建DNS A record并指向 Ingress VIP
- ✓ 更新Ingress状态

为什么需要构建SLB方案

成本

- 硬件LB 价格昂贵
- 固定的tech refresh周期

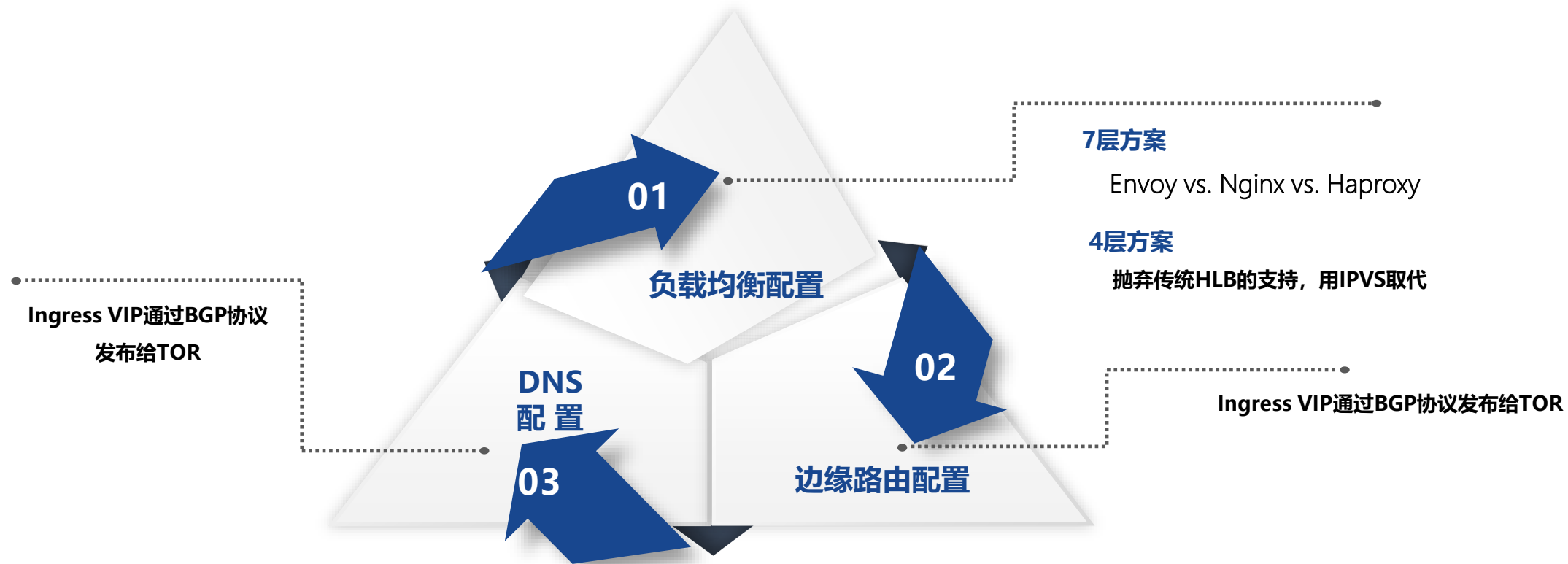
配置管理

- LB设备的onboard/offboard由不同team管理
 - 不同设备API不一样, 不支持L7 API
 - 基于传统的ssh 接口, 效率低下
 - Flex up/down以及Migration复杂

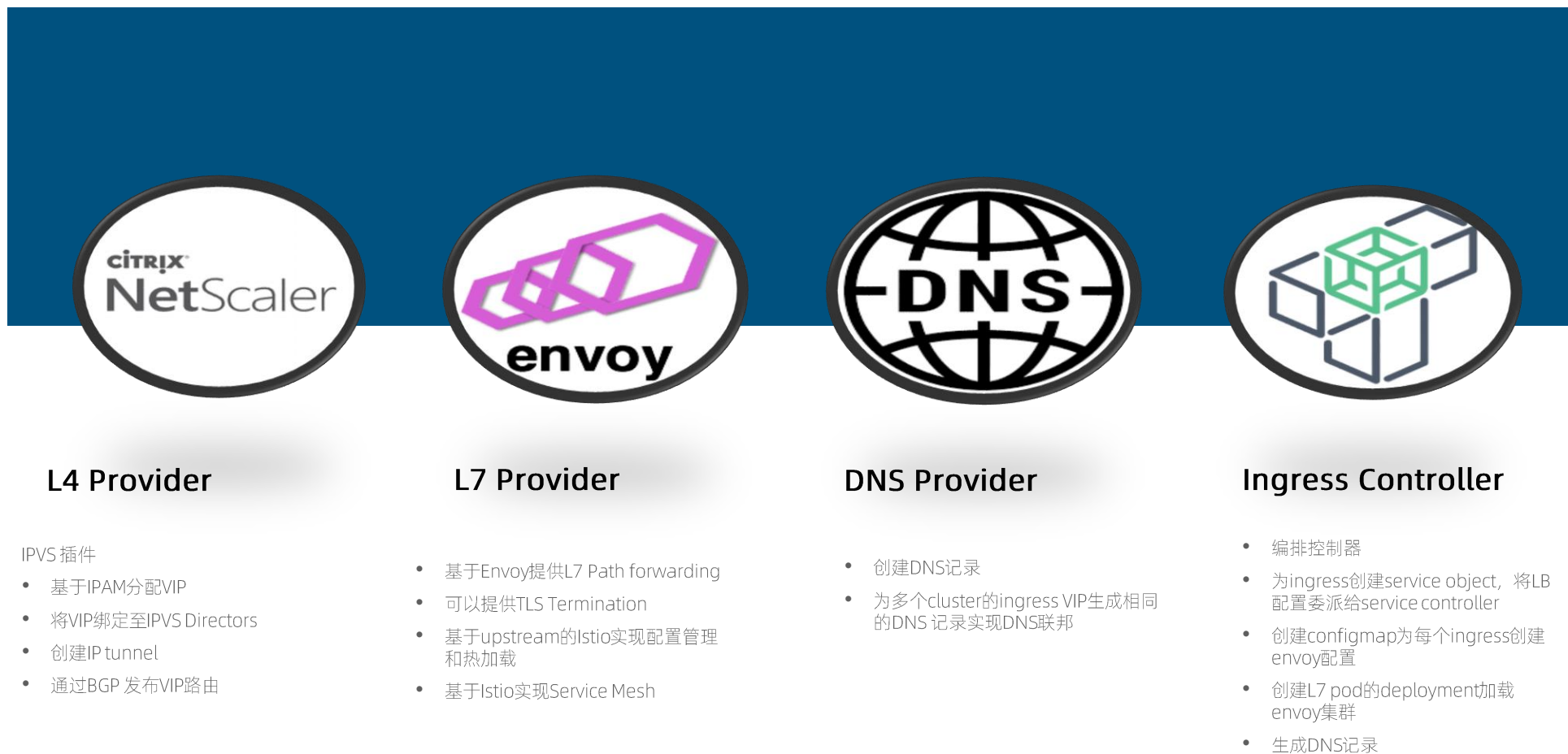
部署模式

- 1+1模式
- 隔离性差

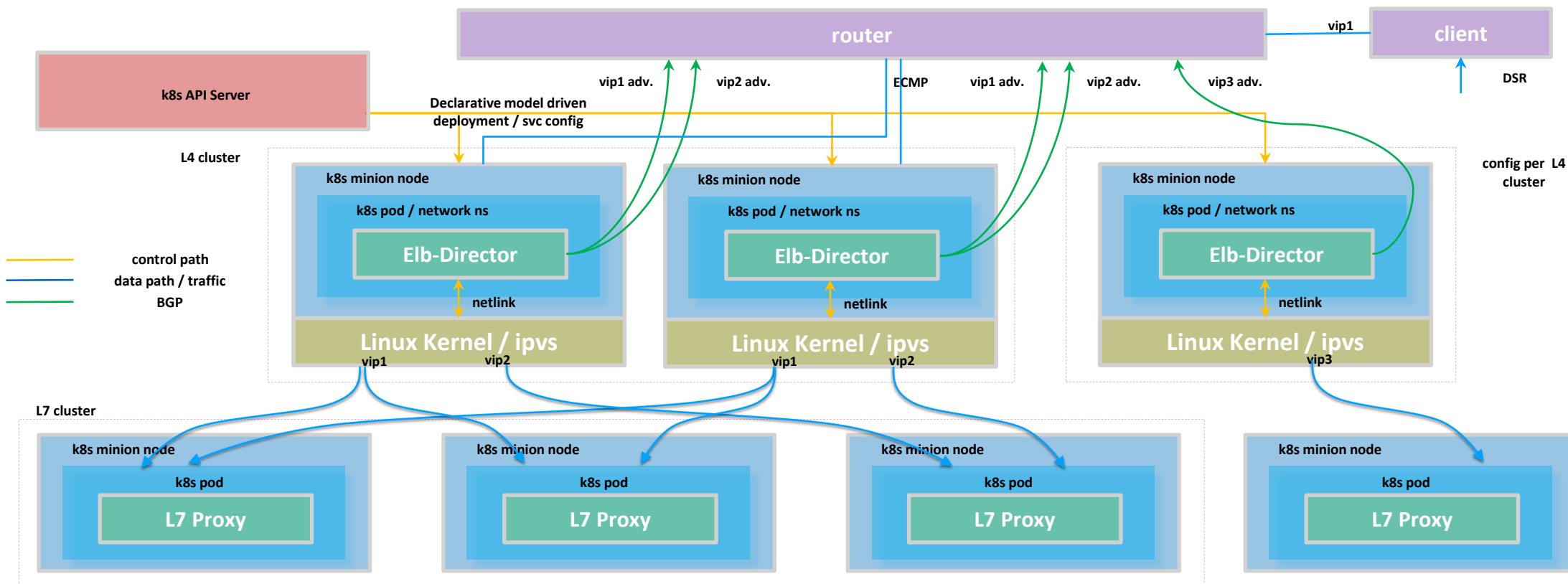
亟待解决的问题



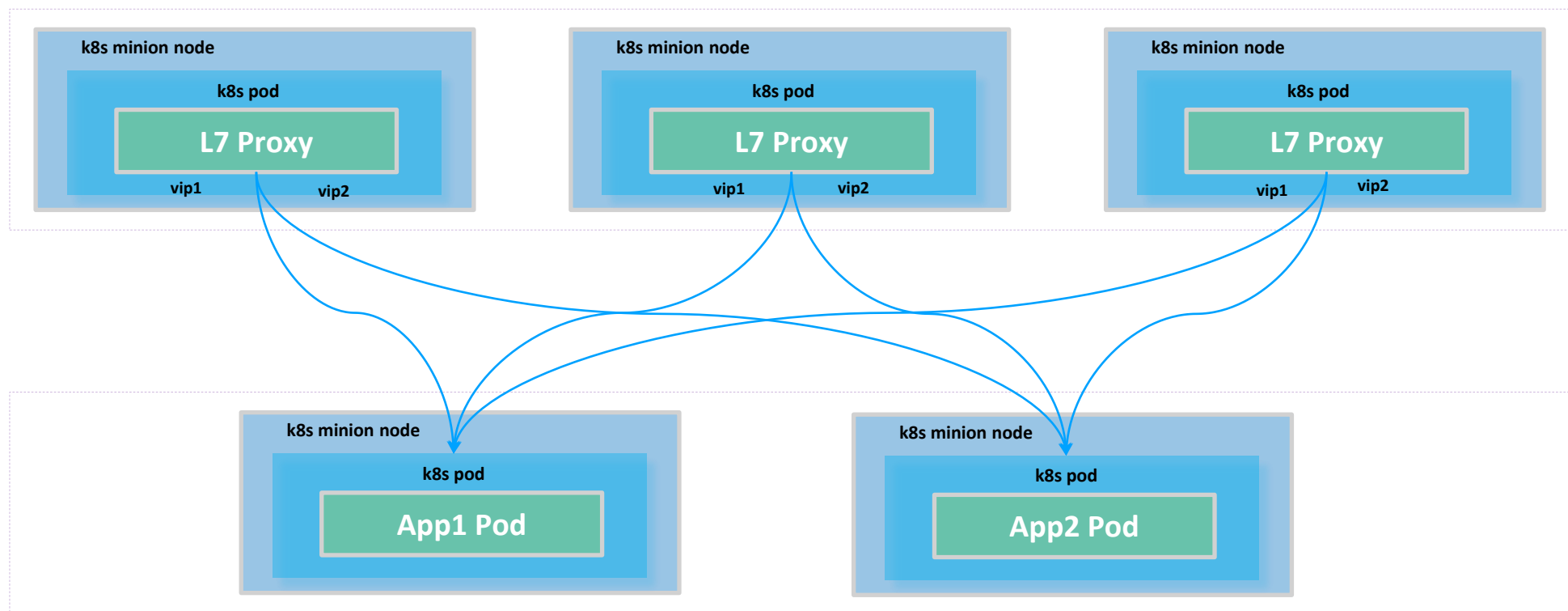
需要解决问题：如何让domain用户自定义后台应用的访问地址，如何优化访问路径



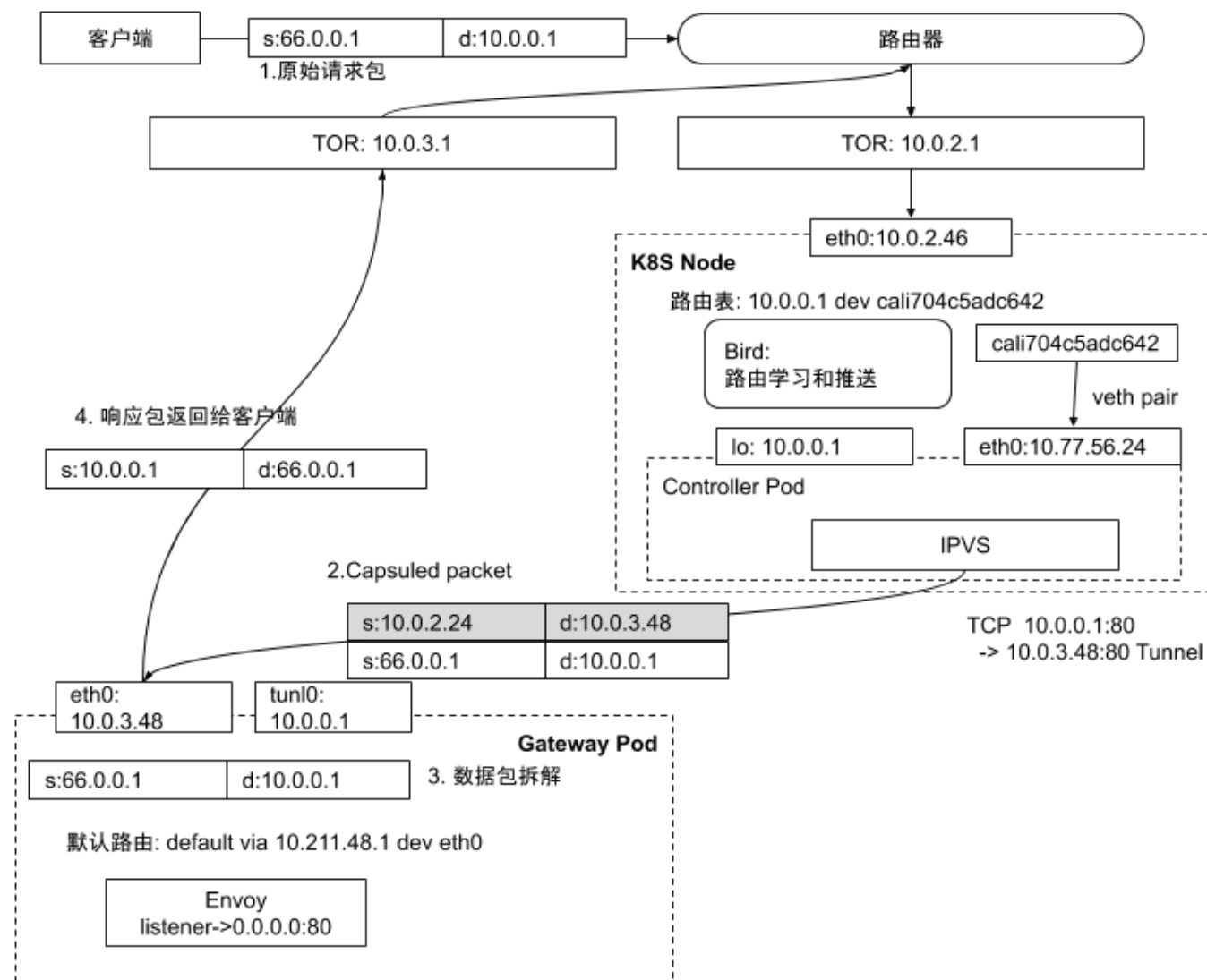
L4 集群架构



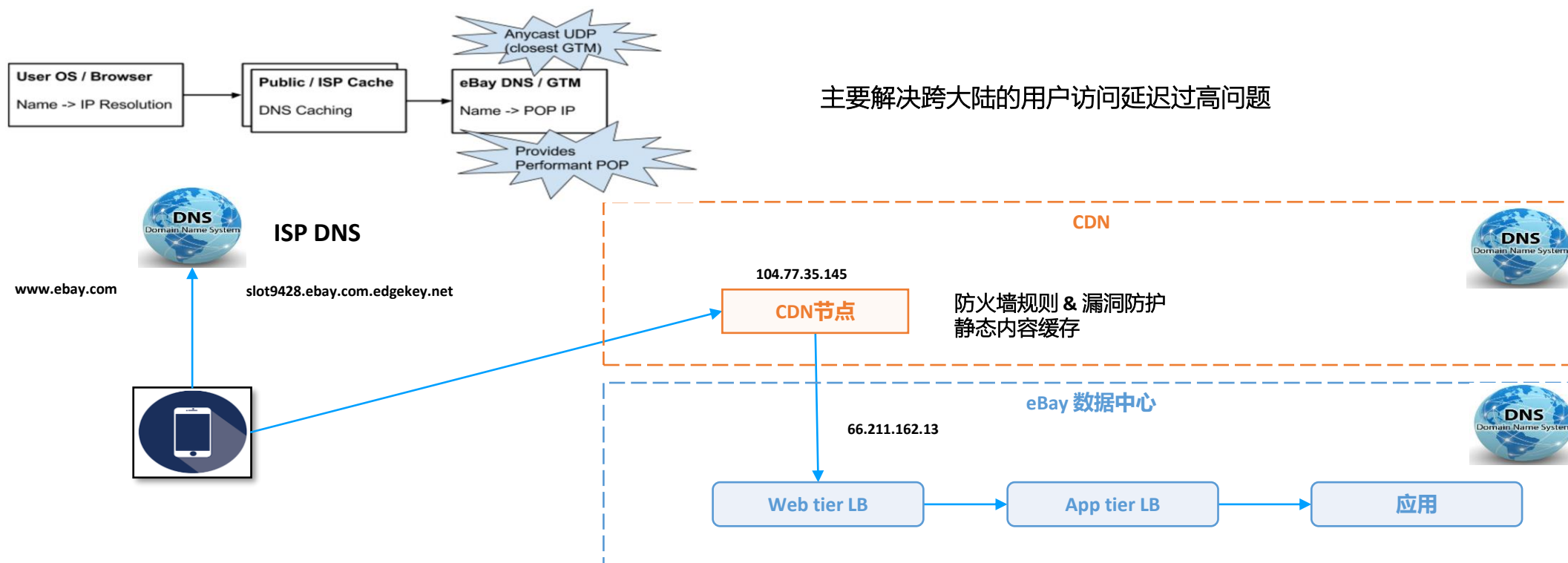
L7 集群架构



数据流



跨大陆的互联网调用



来自eBay的数据显示, 100ms的速度提升能带来0.3%的GMV增长

互联网路径的不确定性



珀斯到美国

Address	AS Number	City	Country	Status
110.141.204.3 [110.141.204.3]	1221	NA	NA	Success
58.162.109.18 [58.162.109.18]	1221	NA	NA	Success
139.130.72.165 [tengigabithernet7-4.pie9.perth.telstra.net]	1221	Perth	Australia	Success
203.50.115.193 [tengigabithernet0-8-0-2.pie-core1.perth.telstra.net]	1221	Perth	Australia	Success
203.50.11.16 [bundle-ether6.way-core4.adelaide.telstra.net]	1221	Adelaide	Australia	Success
203.50.11.93 [bundle-ether9.exi-core10.melbourne.telstra.net]	1221	Melbourne	Australia	Success
203.50.11.124 [bundle-ether12.chw-core10.sydney.telstra.net]	1221	Sydney	Australia	Success
203.50.6.93 [bundle-ether1.oxf-gw11.sydney.telstra.net]	1221	Sydney	Australia	Success
203.50.13.98 [bundle-ether1.sydo-core03.sydney.reach.com]	1221	Sydney	Australia	Success
202.84.222.54 [i-0-1-0-15.sydo-core04.bi.telstraglobal.net]	4637	NA	China	Success
202.84.136.209 [i-0-4-0-29.1wit-core02.bx.telstraglobal.net]	4637	NA	China	Success
202.84.253.82 [i-92.eqia01.bi.telstraglobal.net]	4637	NA	China	Success
134.159.61.26 [isparkie-peer.eqia01.pr.telstraglobal.net]	4637	NA	China	Success
195.22.214.167 [et1-1-0.franco32.fra.seabone.net]	6762	Frankfurt	Germany	Success
89.221.34.133 [ebay.franco32.fra.seabone.net]	6762	Frankfurt	Germany	Success
216.113.190.34 [216.113.190.34]	62955	NA	NA	Success
NA		NA	NA	Timed Out



墨尔本到美国

Address	AS Number	City	Country
168.1.69.243 [f3.45.01a8.ip4.static.sl-reverse.com]	36351	NA	NA
168.1.118.134 [ae12.dar01.mel01.networklayer.com]	36351	Melbourne	Australia
50.97.19.76 [ae8.bbr02.nd01.mel02.networklayer.com]	36351	Melbourne	Australia
50.97.19.64 [ae0.bbr02.eq01.syd02.networklayer.com]	36351	Sydney	Australia
50.97.19.60 [ae7.bbr01.eq01.syd02.networklayer.com]	36351	Sydney	Australia
50.97.19.192 [ae3.bbr01.cs01.lax01.networklayer.com]	36351	Los Angeles	United States
206.223.123.55 [eqix-ix-la.ebay.com]		NA	NA
216.113.190.142 [216.113.190.142]	62955	NA	NA
...		...	...

边缘加速方案综述

- 在全球业务需求较大的城市创建边缘数据中心
 - ✓ 2017年在建阿姆斯特丹，悉尼等8个城市
 - ✓ 迷你型数据中心，每个site15台服务器



- 在边缘数据中心搭建Kubernetes 集群

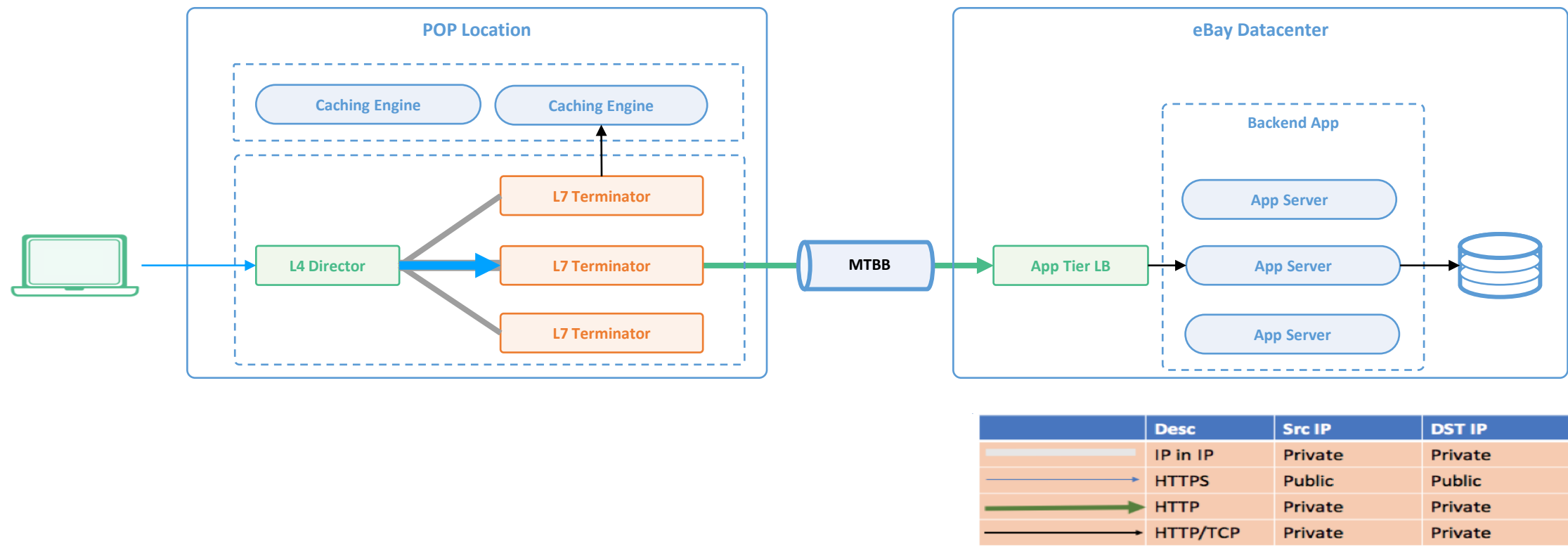


- 部署纯软件的ingress 组件
 - ✓ 启动Ingress Controller完成Ingress 配置
 - ✓ IPVS负责VIP绑定，4层负载均衡以及一致性哈希
- ✓ Envoy把TLS terminate在边缘节点，并提供L7 路由转发
 - ✓ BGP协议将VIP 发布给路由器

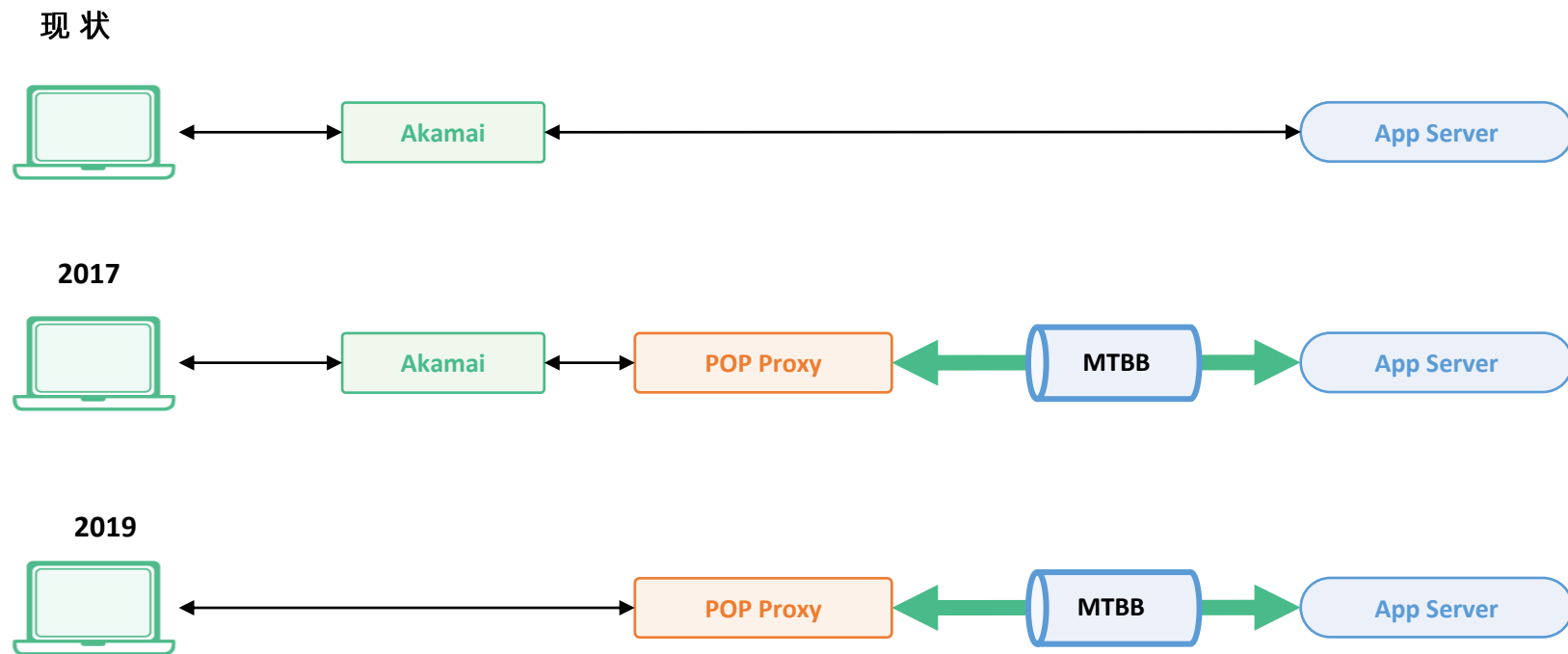


- 规划Ingress 规则
 - ✓ 手工创建endpoint指向eBay数据中心的应用
 - ✓ 创建Ingress并指向endpoint对应的service

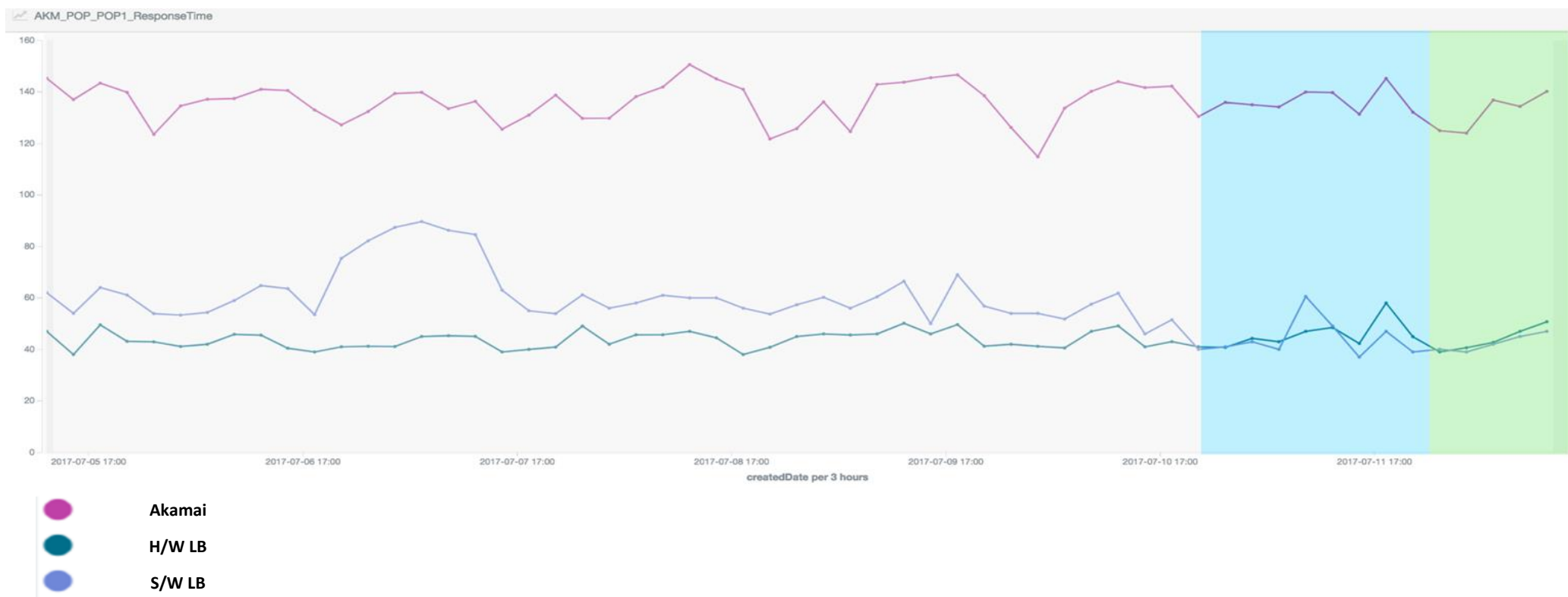
边缘加速组件



对网络路径的优化



不同方案的响应时间对比



课后作业

现在你对Kubernetes的控制面板的工作机制是否有了深入的了解呢？

是否对如何构建一个优雅的云上应用有了深刻的认识，那么接下来用最近学过的知识把你之前编写的http以优雅的方式部署起来吧，你可能需要审视之前代码是否能满足优雅上云的需求。

作业要求：编写Kubernetes 部署脚本将httpserver部署到kubernetes集群，以下是你可以思考的维度

- 优雅启动
- 优雅终止
- 资源需求和QoS保证
- 探活
- 日常运维需求，日志等级
- 配置和代码分离

课后作业（第二部分）

除了将httpServer 应用优雅的运行在Kubernetes之上，我们还应该考虑如何将服务发布给对内和对外的调用方。

来尝试用Service, Ingress将你的服务发布给集群外部的调用方吧

在第一部分的基础上提供更加完备的部署spec，包括（不限于）

- Service
- Ingress

可以考虑的细节

- 如何确保整个应用的高可用
- 如何通过证书保证httpServer的通讯安全

THANKS

 极客时间 | 训练营